



UNIVERSITETI POLITEKNIK – TIRANË
FAKULTETI I TEKNOLOGJISË SË INFORMACIONIT
DEPARTAMENTI I INXHINIERISË INFORMATIKE
SHESHI NËNË TEREZA, 1 – TIRANË
TEL/FAX : +355 4 2278 159

**Identifikimi i Entiteteve dhe Ndikimi i Pjesëve të Ligjëratës në Tekste në
Gjuhën Shqipe duke përdorur Teknikat e Inteligjencës Artificiale**

TEZË PËR FITIMIN E DIPLOMËS BACHELOR
NË
INXHINIERI INFORMATIKE

NGA

Piro Gjikdhima

UDHËHEQËS SHKENCOR

MSc. Alba Haveriku

TIRANË, SHTATOR 2025



UNIVERSITETI POLITEKNIK I TIRANËS
Fakulteti i Teknologjisë së Informacionit
Sheshi Nënë Tereza, 1 – Tiranë
Tel/Fax : +355 4 2278 159

PROJEKT – DIPLOMË
Cikli i Parë i Studimeve
Bachelor
në
Inxhinieri Informatike

TEMA: IDENTIFIKIMI I ENTITETEVE DHE NDIKIMI I
PJESËVE TË LIGJERATËS NË TEKSTE NË GJUHËN SHQIPE
DUKE PËRDORUR TEKNIKAT E INTELIGJENCËS ARTIFICIALE

DEKANI

PËRGJEGJËSI I DEPARTAMENTIT

UDHËHEQËSI

DIPLOMANTI

Prof.Dr. Elinda Meçe

Prof.Asoc. Hakik Paci

Msc. Alba Haveriku

Piro Gjikhima

Tiranë 2024-2025

**REPUBLIKA E SHQIPËRISË
UNIVERSITETI POLITEKNIK I TIRANËS
FAKULTETI I TEKNOLOGJISË SË INFORMACIONIT
DEPARTAMENTI I INXHINIERISË INFORMATIKE**
Sheshi “Nënë Tereza”, Nr. 1, Tiranë
Tel. dhe Fax : (+355) 4 2278 159

DEKANI

Prof.Dr. Elinda Meçe

FLETË – DETYRË

**mbi PROJEKT – DIPLOMËN
Cikli i Parë i Studimeve
Bachelor
në
Inxhinieri Informatike**

Studenti : Piro Artur Gjikhima Nr. i Regj. PO523A110351
(emri, atësia, mbiemri)

Departamenti i Inxhinierisë Informatike

I. Tema e Projekt – Diplomës

Identifikimi i Entiteteve Dhe Ndikimi i Pjesëve Të Ligjeratës në Tekste në Gjuhën Shqipe
duke përdorur Teknikat e Inteligjencës Artificiale

II. Afati i dorëzimit të Projekt- Diplomës 22.09.2025

III. Të dhëna mbi Projekt- Diplomën

1. Gjuha e programimit e përdorur në të gjithë projektin është Python
2. Mjetet për pre-processing: Pandas, NumPy, re, regex, conllu
3. Teknologjitë e përdorura janë: PyTorch, Transformers, Pandas, NumPy, Scikit-learn, Matplotlib etj
4. Platformat dhe mjetet: Google Colab, Hugging Face Transformers, NLTK
5. Modelet e implementuar: Modelet e Thjeshta, BiLSTM, BiLSTM-CRF, BERT, RoBERTa
6. Libraritë për vlerësim: Segeval, Scikit-learn metrics
7. Formatet e të dhënave: CSV, JSON, CoNLL-2003
8. Mjediset e zhvillimit: Jupyter Notebook, Google Colab Pro

IV. Përmbajtja e Projekt- Diplomës

1. Hyrja _____
2. Përpunimi i Gjuhës Natyrore _____
3. Përgatitja e Datasetit _____
4. Algoritmat e Përdorur _____
5. Eksperimentet _____
6. Përfundime _____

V. Kontrollori në Departament (studenti është i detyruar që me materialet e përgatitura në atë kohë të paraqitet në Departament)

1. 13.03.2025 Kontrollori Msc. Alba Haveriku
2. 03.09.2025 Kontrollori Msc. Alba Haveriku
3. 22.09.2025 Kontrollori Prof. Asoc. Hakik Paci

Udhëheqësi _____ Msc. Alba Haveriku
(titulli, emri, mbiemri)

Data e miratimit të temës 13.03.2025

Përgjegjësi i Departamentit

Prof.Asoc. Hakik Paci

Kjo fletë-detyrë plotësohet në dy kopje, një i bashkëngjitet Projekt-Diplomës së kryer, që bashkë me të paraqitet në Komisionin e Mbrojtjes së Projekt-Diplomave dhe tjetra i jepet studentit.

Deklaratë mbi plagjiaturën

Unë konfirmoj se ky material është puna ime individuale, e pa kopjuar nga puna e askujt tjetër (e publikuar ose jo), dhe nuk është dorëzuar për vlerësim më parë në Universitetin Politeknik të Tiranës apo diku tjetër. Unë konfirmoj se njoh Rregulloren e UPT dhe Kodin e Etikës si dhe masat e parashikuara në to për plagjiaturën.

Universiteti Politeknik i Tiranës dhe Udhëheqësi i Diplomës kanë të drejtën e publikimit dhe përdorimit të këtij materiali.

Dt. 22.09.2025

Piro Gjikdhima

Firma

Abstrakt (Shqip)

Ky studim trajton sfidat e Identifikimit të Entiteteve të Emërtuara (NER) dhe Etiketimit të Pjesëve të Ligjëratës (POS) në gjuhën shqipe, ku mjete teknologjike janë ende të kufizuara. Për të adresuar këtë problem, u krijua një dataset gjithëpërfshirës, i cili kombinon etiketimin për NER dhe POS duke zgjeruar dhe pastruar datasetin ekzistues NER me etiketat POS. Duke përdorur datasetin e krijuar janë ndërtuar modele etiketuese NER duke trajnuar algoritme të ndryshme të Inteligjencës Artificiale, përfshirë algoritme tradicionale, BiLSTM dhe transformer. Në një pjesë të modeleve të trajnuar janë përdorur dhe informacion shtesë për etiketat POS dhe temat e fjalëve. Rezultatet tregojnë se modelet transformer performojnë më mirë se BiLSTM dhe modelet tradicionale, veçanërisht kur përfshihen veçoritë shtesë, duke ofruar një saktësi dhe qëndrueshmëri më të madhe në etiketimin NER. Ky studim ofron një burim të ri për gjuhën shqipe dhe demonstroi se kombinimi i etiketave NER me POS dhe temat e fjalëve dhe përdorimi i zgjidhjeve teknologjike moderne përmirësojnë ndjeshëm performancën e modeleve të përpunimit të gjuhës natyrale, duke hapur rrugën për zhvillime të mëtejshme në aplikacione inteligjente dhe kërkime të avancuara në përpunimin e gjuhës shqipe.

Abstract (English)

This study addresses the challenges of Named Entity Recognition (NER) and Part-of-Speech (POS) tagging in the Albanian language, where technological tools are still limited. To address this problem, a comprehensive dataset was created, which combines labeling for NER and POS by expanding and cleaning the existing NER dataset with POS labels. Using the created dataset, NER labeling models were built by training different Artificial Intelligence algorithms, including traditional algorithms, BiLSTM and transformer. In some of the trained models, additional information for POS labels and word topics was also used. Results show that transformer models perform better than BiLSTM and traditional models, especially when additional features are included, offering greater accuracy and consistency in NER labeling. This study provides a new resource for the Albanian language and demonstrates that combining NER labels with POS and word topics and using modern technological solutions significantly improve the performance of natural language processing models, paving the way for further developments in intelligent applications and advanced research in Albanian language processing.

Falënderime

Dëshiroj të shpreh mirënjohjen dhe falënderimet e mia më të sinqerta për MSc. Alba Haveriku, udhëheqësen e këtij punimi diplome, për mbështetjen e vazhdueshme, këshillat e çmuara dhe inkurajimin e pandërprerë gjatë gjithë realizimit të këtij projekti. Ekspertiza e saj në fushën e teknologjive moderne dhe sugjerimet e saj të dobishme kanë luajtur një rol vendimtar në formësimin e drejtimit dhe cilësisë së punimit.

Jam veçanërisht mirënjohës për kohën dhe energjinë që ka dhënë, si dhe për gatishmërinë e saj për të ofruar udhëzim akademik dhe profesional në çdo etapë të zhvillimit të këtij projekti. Përkushtimi, durimi dhe mbështetja e saj kanë qenë të çmuara gjatë gjithë procesit të hulumtimit dhe implementimit, dhe e vlerësoj sinqerisht kontributin e saj të vyer në suksesin e punimit.

Përmbajtja

<i>Abstrakt (Shqip)</i>	5
<i>Abstract (English)</i>	6
<i>Falënderime</i>	7
<i>Përmbajtja</i>	8
<i>Lista E Tabelave</i>	11
<i>Lista E Figurave</i>	12
<i>Shkurtime</i>	13
<i>KREU I</i>	14
<i>HYRJE</i>	14
1.1. PËRMBLEDHJE E PËRGJITHSHME	14
1.1.1. <i>Motivi</i>	14
1.1.2. <i>Objektivat</i>	15
<i>KREU II</i>	16
<i>PËRPUNIMI I GJUHËS NATYRORE</i>	16
2.1. <i>NER</i>	16
2.1.1. <i>Përkufizimi dhe Rëndësia e NER</i>	16
2.1.2. <i>Skemat e Etiketimit për NER</i>	16
2.2 <i>PJESËT E LIGJËRATËS - POS</i>	17
2.2.1. <i>Përkufizimi dhe Rëndësia e POS</i>	17
2.2.2. <i>UPOS</i>	17
<i>KREU III</i>	18
<i>PËRGATITJA E DATASET-IT</i>	18
3.1. <i>DATASET-I I NER</i>	18
3.2. <i>DATASET-I I POS</i>	18
3.3. <i>DATASET-I I PLOTË</i>	19
3.3.1 <i>Sfida Kryesore të Bashkimit</i>	19
3.3.2 <i>Metodologjia e Bashkimit</i>	20
3.3.3 <i>Optimizimi i Dataset-it Final</i>	22
<i>KREU IV</i>	23
<i>ALGORITMAT E PËRDORUR</i>	23
4.1. <i>ALGORITMAT MACHINE LEARNING</i>	23

4.2. BiLSTM.....	23
4.3. BiLSTM-CRF	24
4.4. MODELE SHITESË.....	25
4.4.1. BERT.....	25
4.4.2. RoBERTa	27
KREU V.....	29
EKSPERIMENTET.....	29
5.1. MËNYRA E EKSPERIMENTIT TË MODELEVE TË THJESHTA.....	29
5.1.1. Struktura e programit	29
5.1.2. Metrikat	29
5.1.3. Libraritë dhe Importet	30
5.1.4. Procesi i Përpunimit të të Dhënave.....	30
5.1.5. Strukturimi i të Dhënave për Trajnim	30
5.1.6. Procesi i Trajnimit dhe Vlerësimit	31
5.1.7. Implementimi i MPL.....	31
5.1.8. Ekzekutimi dhe Mbledhja e Rezultateve	32
5.2. MËNYRA E EKSPERIMENTIT TË BiLSTM-VE.....	33
5.2.1. Libraritë dhe Importet	33
5.2.2. Procesi i Përpunimit të të Dhënave.....	33
5.2.3. Arkitekturat e Modeleve.....	33
5.2.3.1. BiLSTM i Thjeshtë	33
5.2.3.2. BiLSTM me Veçori POS	34
5.2.3.3. BiLSTM me Veçori të Plota	35
5.2.3.4. BiLSTM-CRF i Thjeshtë.....	36
5.2.3.5. BiLSTM-CRF me Veçori POS.....	37
5.2.3.6. BiLSTM-CRF me Veçori të Plota	37
5.2.4. Menaxhimi i Veçorive.....	39
5.2.5. Procesi i Trajnimit dhe Early Stopping	39
5.2.6. Ekzekutimi dhe Optimizimi	40
5.3. MËNYRA E EKSPERIMENTIT TË BERT DHE ROBERTA.....	40
5.3.1. Libraritë dhe Importet	40
5.3.2. Parapërgatitja e të Dhënave dhe Hartimi i Etiketave.....	41
5.3.3. Arkitekturat	41
5.3.4. Struktura e Trajnimit dhe Optimizimi	43
5.3.5. Ndryshimet Arkitekturore midis BERT dhe RoBERTa	43
5.3.6. Procesi i Trajnimit dhe Krahimit	43
5.4. REZULTATET.....	44
5.4.1. Rezultatet e Modeleve Machine Learning	44
5.4.2. Rezultatet e BiLSTM-ve.....	45
5.5.3. Rezultatet e Transformerave	46
KREU VI.....	48

<i>PËRFUNDIME</i>	48
Bibliografia	49

Lista e Tabelave

Tabela 2.1. Skema e Etiketave NER

Tabela 2.2. Skema e etiketave POS

Tabela 5.2 Parametra të modeleve

Tabela 5.2 Rezultatet e Modeleve Machine Learning

Tabela 5.3 Rezultatet e Modeleve BiLSTM

Tabela 5.4 Rezultatet e Modeleve Transformer

Lista e Figurave

Figura 2.1. Dataset-i i NER

Figura 2.2. Dataset-i i POS

Figura 3.1. Paraqitje e BiLSTM

Figura 3.2. Model i BERT

Figura 5.1. Arkitektura e BiLSTM të Thjeshtë

Figura 5.2. Arkitektura e BiLSTM me POS

Figura 5.3. Arkitektura e BiLSTM (LEMMA + POS)

Figura 5.4. Arkitektura e BiLSTM-CRF të Thjeshtë

Figura 5.5. Arkitektura e BiLSTM-CRF (POS)

Figura 5.6. Arkitektura e BiLSTM-CRF (POS + LEMMA)

Figura 5.7. Arkitektura e BERT

Figura 5.8. Arkitektura e RoBERTa

Figura 5.9. Rezultati i Parashikimeve

Shkurtime

NLP – Natural Language Porcessing

NER – Named Entity Recognition

POS – Part Of Speech

BIO - Beginning, Inside, Outside

UPOS - Universal POS Tags

UD - Universal Dependencies

RNN – Recurrent Neural Network

BiLSTM – Bidirectional Long Short-Term Memory

BERT – Bidirectional Encoder Representations from Transformers

MLM – Masked Language Modeling

NSP – Next Sentence Prediction

RoBERTa – Robustly Optimized BERT Approach

MISC – Miscellaneous

NB – Naive Bayes

ET – Extra Trees

RF – Random Forest

GB – Gradient Boosting

XGBoost – Extreme Gradient Boosting

MLP – Multi-Layer Perceptron

CRF – Conditional Random Fields

CSV – Comma Separated Values

KREU I

HYRJE

1.1. Përmbledhje e përgjithshme

Përpunimi i gjuhës natyrore (NLP) përfaqëson një nga fushat më dinamike dhe me rritje të shpejtë të inteligjencës artificiale, duke kombinuar principet e informatikës, linguistikës kompjuterike dhe matematikës për të mundësuar makinave të kuptojnë, interpretojnë dhe gjenerojnë gjuhën njerëzore. Kjo fushë ndërdisiplinore ka gjetur aplikime të gjera në teknologjinë moderne, duke përfshirë përkthimin automatik, analizat sentimentale, ekstraktimin e informacionit, sistemet e pyetje-përgjigje, dhe asistentët virtualë.

Megjithatë, përparimi në NLP nuk ka qenë i njëtrajtshëm për të gjitha gjuhët. Ndërsa gjuhët me burime të bollshme si anglishtja kanë përfituar nga investimet e mëdha në kërkimin shkencor dhe zhvillimin teknologjik, gjuhët me burime të kufizuara, si gjuha shqipe, përballen me sfida të konsiderueshme. Për gjuhën shqipe, e folur nga rreth 7-8 milionë njerëz, mungojnë në mënyrë kritike burimet e standardizuara dhe dataset-et e etiketuara që janë thelbësore për zhvillimin e aplikacioneve të NLP-së.

Kjo mungesë është veçanërisht e dukshme për detyrat themelore të NLP-së si NER dhe POS. NER-i përfshin identifikimin dhe klasifikimin e entiteteve specifike në tekst (si emra personash, vendesh, organizatash), ndërsa POS tagging-u cakton kategorinë gramatikore për çdo fjalë në një fjali. Këto detyra janë komponente kyçe që shërbejnë si hapi i parë për shumë aplikacione të avancuara të NLP-së.

Ky punim synon të krijojë një dataset gjithëpërfshirës për NER dhe POS në gjuhën shqipe, duke përdorur skemën BIO për NER dhe Universal POS Tags për POS dhe duke i bashkuar ato në format CoNLL-U. Përveç ndërtimit të dataset-it, krahasohet performanca e algoritmave të ndryshme, duke përfshirë metoda tradicionale dhe rrjete RNN. Rezultatet raportohen me metrike standarde F1-score.

Ky studim synon të kontribuojë në zhvillimin e burimeve për gjuhën shqipe dhe të krijojë një bazë solide për kërkime dhe aplikacione të mëtejshme në NLP.

1.1.1. Motivi

Përpunimi i gjuhës natyrore (NLP) ka marrë një rëndësi të veçantë në shumë fusha të teknologjisë moderne, duke përfshirë përkthimin automatik, kërkimin e informacionit dhe sistemet inteligjente. Megjithatë, për gjuhën shqipe ekziston një mungesë e dukshme e burimeve të standardizuara, veçanërisht për detyrat bazë si Identifikimi i Entiteteve të Emërtuara (NER) dhe Etiketimi i Pjesëve të Fjalës (POS). Kjo mungesë kufizon zhvillimin

e aplikacioneve të avancuara të NLP dhe kërkon krijimin e dataset-eve të pastruara dhe të strukturuar në mënyrë të besueshme.

1.1.2. Objektivat

Ky punim synon të arrijë objektivat kryesore të mëposhtme:

1. Të krijojë një **dataset gjithëpërfshirës për NER dhe POS tagging** në gjuhën shqipe, duke përdorur skemën BIO për NER dhe Universal POS Tags për POS.
2. Të trajnojë dhe të vlerësojë **modele të ndryshme** për këto detyra, duke përfshirë metoda tradicionale, BiLSTM dhe modele Transformer (BERT, RoBERTa).
3. Të analizojë performancën e modeleve duke përdorur metrikat standarde **F1-score** për rezultate të qëndrueshme.

Realizimi i këtyre objektiveve do të kontribuojë në mënyrë të konsiderueshme në zhvillimin e ekosistemit të NLP-së për gjuhën shqipe.

KREU II

PËRPUNIMI I GJUHËS NATYRORE

2.1. NER

2.1.1. Përkufizimi dhe Rëndësia e NER

Identifikimi i Entiteteve të Emërtuara (NER) përbën një nga detyrat më themelore dhe me rëndësi kritike në përpunimin e gjuhës natyrore, duke shërbyer si një komponent kyç që mundëson kuptimin e strukturuar të informacionit tekstual. NER-i synon të identifikojë dhe të klasifikojë në mënyrë automatike entitetet e emërtuara që përmenden në tekst, duke i kategorizuar ato në klasa të paracaktuara semantike. NER është procesi i identifikimit të sekuencave të fjalëve në tekst që përfaqësojnë entitete specifike të botës reale dhe caktimi i një etikete kategorike për secilën prej tyre. Tabela 2.1 paraqet skemën e anotimit për NER.

2.1.2. Skemat e Etiketimit për NER

Skema BIO (Begin-Inside-Outside)

Skema BIO është një nga metodat më të përdorura për etiketimin e entiteteve në NER. Kjo skemë përdor tre lloje etiketash:

- **B- (Begin):** Tregon fillimin e një entiteti të ri
- **I- (Inside):** Tregon vazhdimin e një entiteti që ka filluar më parë
- **O (Outside):** Tregon se token-i nuk është pjesë e asnjë entiteti

Për skemat e anotimit të NER në gjuhën shqipe, në këtë studim është adoptuar skema e propozuar nga [1]. Siç paraqitet në punimin e tyre Tabela 1.1:

Tabela 1.1 Skema e etiketave NER

Labels	Category	Explanation
PER	Person	name and surname, only name, only surname initials
ORG	Organization	political parties, government structures, departments, federations, forums, agencies, authorities, associations, foundations, etc.
PRO	Product	vaccines, objects, etc.
EVENT	Event	conferences, summits, conventions, congresses, championships, etc.
VEND_0	City	names of cities
VEND_1	Country	names of countries
SHESH	Square	names of squares
RRUGE	Street	names of streets
DATE_0	Date	full dates, only year, only month and year, only day and month
DATE_1	Days of the week	days of the week

Kjo skemë anotimi është zgjedhur për shkak të përshtatshmërisë së saj me karakteristikat morfosintaksore të gjuhës shqipe dhe mundëson një klasifikim të detajuar të entiteteve të emërtuara që hasen më shpesh në tekstet shqipe.

2.2 Pjesët e ligjëratës - POS

2.2.1. Përkufizimi dhe Rëndësia e POS

Etiketimi i Pjesëve të Fjalës është procesi i caktimit të kategorisë gramatikore për çdo fjalë në një tekst. Ky proces është thelbësor për analizën sintaksore dhe semantike të gjuhës, duke shërbyer si bazë për shumë aplikacione të tjera të NLP si analizimi sintaksor, përkthimi automatik dhe sisteme të përgjigjes ndaj pyetjeve.

2.2.2. UPOS

Projekti Universal Dependencies (UD) ka zhvilluar një set universal etiketash POS që mund të aplikohen në shumë gjuhë, duke përfshirë gjuhën shqipe. Këto etiketa sigurojnë një standard ndërkombëtar për krahasimin dhe analizën e gjuhëve të ndryshme.

Për skemat e POS në gjuhën shqipe, në këtë studim është adoptuar skema e propozuar nga [2]. Siç paraqitet në punimin e tyre Tabeala 1.2.

Tabela 2.2. Skema e etiketave POS

Part of speech	POS tag	Features
Noun	NOUN	Case Definite Gender Number
Proper noun	PROPN	
Adjective	ADJ	Degree
Numerals	NUM	NumType
Pronoun	PRON	Case Gender Number Person PronType
Verb	VERB / AUX	Mood Number Person Tense VerbForm
Adverb	ADV	AdvType
Preposition	ADP	
Conjunction	CCONJ / SCONJ	
Part	DET	
Interjection	INTJ	
Symbol	SYM	
Punctuation	PUNCT	

KREU III

PËRGATITJA E DATASET-IT

3.1. Dataset-i i NER

Dataset-i NER i përdorur në këtë punim diplome është marrë nga [1]. Ai quhet korpus.txt dhe, pas disa rregullimeve të vogla manuale, numri i token-ëve arriti në 1,023,483 (39226 fjali) nga 1,003,836.

Ky dataset përmban struktura të etiketimit të entiteteve sipas standardeve BIO dhe është përdorur si bazë për trajnimin e modeleve. Një paraqitje e përmbajtjes së tij jepet në Figurën 2.1:

biodiversitetit	0
,	0
arkeologjisë	0
,	0
etnografisë	0
dhe	0
historisë	0
cicëroni	0
Liridon	B-PER
Cena	I-PER
thotë	0
se	0
e	0
prezantojnë	0
mjaft	0
mirë	0
Kukësin	B-VEND_0
.	0

Figura 3.1. Dataset-i i NER

Ky është dataset-i më i saktë në dispozicion, pasi është përpunuar manualisht dhe përbën themelin e bashkimit të dy datasetëve në një strukturë unike dhe të unifikuar. Kjo e bën atë referencën kryesore për ndërtimin e modelit dhe vlerësimin e rezultateve.

3.2. Dataset-i i POS

Dataset-i i POS i përdorur në këtë punim diplome është marrë nga [2]. Për secilin tekst është gjeneruar një file në formatin *CoNLL-U*, brenda të cilit është kryer analiza morfologjike e fjalëve.

Ky format mundëson ruajtjen e informacionit gjuhësor, si rrënja, POS tag-u, dhe veçoritë morfologjike, në një strukturë të standardizuar, e cila përdoret gjerësisht në përpunimin e gjuhës natyrore. Një paraqitje e përmbajtjes së tij jepet në Figurën 2.2 :

```
# text = Vendi ynë, pas hedhjes së shortit, është përcaktuar se do të jetë i 6-ti në radhë, të enjten e 9 majit 2024, kur do të zhvillohet edhe gjysmëfinalja e dytë.
# sent_id = 1
1 Vendi vend NOUN _ Case=Nom|Definite=Def|Gender=Masc|Number=Sing 10 nsubj _ start_char=205|end_char=210
2 ynë ynë PRON _ Case=Nom|Gender=Masc|Number=Sing|Person=1|Poss=Yes|PronType=Prs 1 flat _ start_char=211|end_char=214|SpaceAfter=No
3 , , PUNCT _ _ 5 punct _ start_char=214|end_char=215
4 pas pas ADP _ Case=Abl 5 case _ start_char=216|end_char=219
5 hedhjes hedhje NOUN _ Case=Abl|Definite=Def|Gender=Fem|Number=Sing 10 advmod _ start_char=220|end_char=227
6 së së DET _ Case=Abl|Gender=Fem|Number=Sing|PronType=Art 7 det:poss _ start_char=228|end_char=230
7 shortit short NOUN _ Case=Gen|Definite=Def|Gender=Masc|Number=Sing 5 nmod _ start_char=231|end_char=238|SpaceAfter=No
8 , , PUNCT _ _ 5 punct _ start_char=238|end_char=239
9 është jam AUX _ Mood=Ind|Number=Sing|Person=3|Tense=Pres|Voice=Act 10 aux _ start_char=240|end_char=245
10 përcaktuar përcaktoj VERB _ VerbForm=Part 0 root _ start_char=246|end_char=256
11 se se SCONJ _ _ 14 mark _ start_char=257|end_char=259
12 do do PART _ _ 14 fixed _ start_char=260|end_char=262
13 të të PART _ _ 14 fixed _ start_char=263|end_char=265
14 jetë jam VERB _ Mood=Sub|Number=Sing|Person=3|Tense=Pres|Voice=Act 10 advcl _ start_char=266|end_char=270
15 i i DET _ Case=Nom|Gender=Masc|Number=Sing|PronType=Art 16 det:poss _ start_char=271|end_char=272
16 6-ti 6-ti PROPN _ Case=Nom|Definite=Def|Gender=Masc|Number=Sing 14 cop _ start_char=273|end_char=277
17 në në ADP _ Case=Acc 18 case _ start_char=278|end_char=280
18 radhë radhë NOUN _ Case=Acc|Definite=Ind|Gender=Fem|Number=Sing 14 advmod _ start_char=281|end_char=286|SpaceAfter=No
19 , , PUNCT _ _ 21 punct _ start_char=286|end_char=287
20 të të DET _ Case=Acc|Gender=Fem|Number=Sing|PronType=Art 21 det _ start_char=288|end_char=290
```

Figura 3.2. Dataset-i i POS

Ky është dataset-i më i pak saktë sesa dataset-i i iNER, pasi analizat morfologjike dhe etiketimet e POS janë gjeneruar automatikisht dhe nuk janë përpunuar manualisht. Gjatë krijimit të fjalive apo analizës së fjalës, kanë ndodhur gabime dhe gjithësej janë krijuar 1360862 (58651 fjali) token që është vlerë shumë më e madhe se dataset-i i iNER. Megjithatë, ai mbetet i dobishëm për trajnimin e modeleve përkatëse të përpunimit të gjuhës natyrore, veçanërisht për parashikimin e pjesëve të fjalës dhe veçorive morfologjike.

Përdorimi i këtij dataset-i së bashku me dataset-in iNER lejon krijimin e një baze më të plotë dhe të integruar, ku mund të trajnohen modellet për detektimin e entiteteve të emërtuara dhe analizën gramatikore të fjalëve në një kohë.

3.3. Dataset-i i Plotë

3.3.1 Sfida Kryesore të Bashkimit

Gjatë procesit fillestar të bashkimit, u zbulua një problem kritik në përputhjen e fjalëve midis dataset-it iNER dhe POS. Fjalët shpesh nuk përputheshin drejtpërdrejt për shkak të:

- Diferenca në tokenizimin e fjalëve të përbëra
- Karaktere të veçanta pikësimi që ndaheshin ndryshe
- Datat që ishin tokenizuar gabim

Për më tepër, nuk dihej rendi i fjalëve dhe fjalive, pasi dataset-i iNER ishte një file i tërë, kurse dataset-i POS përbëhej nga shumë file që analizonin tekste të ndryshme. U gjet

rendi i gjithë fjalëve dhe fjalive duke rindërtuar fjalitë prej dataset-it NER dhe duke i krahasuar me dataset-in POS. Këto u ruajtën në disa direktori të ndryshme sipas rradhës së tyre në NER.

3.3.2 Metodologjia e Bashkimit

Hapi i parë ishte normalizimi i karaktereve të veçanta, veçanërisht shenjave të pikësimit dhe citimeve. Ky proces siguron që të gjitha variantet e karaktereve të veçantë të trajtohen në mënyrë konsistente gjatë procesit të bashkimit. Shembull, mund të përmendim ‘ ’, ‘ ’, ‘ ’, të cilat duhen kthyer në format " apo ', në mënyrë që të lexohen saktë.

Dataset-i NER u ngarkua duke lexuar skedarin kryesor që përmbante të gjitha fjalët e etiketuara me informacionin përkatës NER. Ky skedar përmban mijëra fjalë të organizuara në format të thjeshtë tekstual, ku çdo rresht përfaqëson një fjalë të shoqëruar me etiketën e saj NER dhe një hapësirë kur fjalja mbaronte.

Dataset-i POS u ngarkua duke kërkuar dhe përpunuar të gjithë skedarët CoNLL-U të shpërndarë në dhjetë direktoritë e ndryshme. Ky proces u krye në mënyrë paralele për të optimizuar performancën, duke marrë parasysh madhësinë e konsiderueshme të dataset-it. Secili skedar CoNLL-U përmban informacion të detajuar gramatikor për çdo fjalë, duke përfshirë formën origjinale, lemën, etiketën gramatikore, dhe marrëdhëniet sintaksore.

Çdo fjali në të dyja dataset-et u caktua një identifikues unik për të mundësuar gjurmimin dhe organizimin e të dhënave gjatë procesit të bashkimit. Kjo organizim ishte kritike për të siguruar që konteksti i fjalive të ruhej gjatë përpunimit.

Identifikuesit u krijuan në mënyrë sekuenciale, duke filluar nga 1 dhe duke vazhduar në rend kronologjik sipas paraqitjes së fjalive në dataset-et origjinale. Kjo qasje siguron që struktura origjinale e tekstit të ruhet sa më shumë të jetë e mundur.

Një problem i madh që u has ishte ngjitja e shenjave të pikësimit para ose pas fjalëve. Për shembull, fjala erdha kishte formë “ erdha. ” ose ‘ erdha’ ’. Këto fjalë me të tilla gabime e vështirësonin lidhjen me pjesën e POS. Gjithashtu dhe POS vuante nga probleme të tilla. Në mënyrë që të rregullohej ky problem, dataset-i i NER u përpunua përsëri duke ndarë këta token me të tilla probleme, me anë të një shprehje regex. Duke qenë se dataset-i NER ishte më i pastër, ai ishte baza e bashkimit.

Fillimisht u gjenden tekstet prej së cilës janë marrë të dhënat NER dhe POS. U organizuan në strukture të njëjtë si file-t POS. U vu re që një pjesë e fjalive të teksteve nuk bënë pjesë në analizën NER dhe kjo e bënte shumë më të vogël dataset-in NER sesa POS. U krye një proces i dyfishtë tokenizimi për të siguruar përputhje optimale midis dy dataset-eve.

Tokenizimi fillestar përfshinte ndarjen e teksteve të thjeshta bazuar në hapësira dhe karaktere të veçantë, duke krijuar njësi individuale që mund të përputheshin me korpusin NER. Përputhja bëhej një-me-një ose në të kundërt vendosej tag-u O. Një gjë e tillë u bë që të pasqyronte me tokenizim NER tekstet e thjeshta që përkonin me të njëjtin tekst të analizuar POS, në mënyrë që analiza dhe bashkimi të bëhej për tekstet e njëjta dhe mos të përdorej më dataset-i i madh NER. Një gjë e rëndësishme është që rendi i fjalëve është ruajtur në çdo tekst.

Tokenizimi i dytë u zhvillua me të njëjtin proces. Duke qenë se fjalët në tekste ishin tokenizuar dhe përkonin një-me-një me fjalët në NER, çdo file u përditësua me fjalët që ishin pjesë e dataset-it dhe të tjerat që nuk bënë pjesë u fshinë. Të gjitha filet u përditësuan dhe kishim një numër dhe strukturë identike të NER dhe POS. Kjo gjë beri të mundur që bashkimi të bëhej file-file.

Algoritmi i propozuar për bashkimin e dy burimeve heterogjene të të dhënave është ndërtuar mbi bazat e Programimit Dinamik, duke u përshtatur nga algoritmi klasik **Needleman-Wunsch** [8], i cili gjen përdorim të gjerë në bioinformatikë për përputhjen e sekuencave biologjike. Në dallim nga qasja tradicionale një-dimensionale, zgjidhja jonë implementon një mekanizëm tri-dimensionale, ku për secilin hap vlerësohen tre alternativa kryesore: (i) kapërcimi i një token-i në dataset-in NER, (ii) kapërcimi i një token-i në dataset-in POS, ose (iii) përputhja e një token-i NER me një ose më shumë token-a nga dataset-i POS. Kjo qasje e zgjeruar është e domosdoshme për trajtimin e diferencave strukturore të krijuara nga tokenizimi i ndryshëm midis dy burimeve.

Një komponent inovativ i algoritmit është përdorimi i span-it adaptiv, i cili mundëson përafrimin dinamik të një token-i nga NER me një numër variabël token-ësh nga POS. Ky span përcaktohet mbi bazën e karakteristikave leksikore të fjalës: fjalët e shkurtra dhe alfanumerike priren të lejojnë span më të gjatë, ndërsa fjalët e gjata dhe komplekse kufizohen në span më të shkurtër. Ky mekanizëm është veçanërisht i rëndësishëm në trajtimin e rasteve të veçanta, siç janë datat. Për shembull, në dataset-in NER, data “20/10/2020” shpërbëhej në token-a individuale “20, /, 10, /, 2020”, ndërkohë që në dataset-in POS, e njëjta njësi paraqitej në formën “20, /10, /202, 0”. Span-i adaptiv lejon krijimin e një përputhjeje optimale ndërmjet këtyre strukturave të ndryshme.

Përveç trajtimit të dallimeve në tokenizim, algoritmi adreson edhe variacionet ortografike nëpërmjet integritit të teknikave të fuzzy matching. Një prag tolerance prej **80%** përdoret për të konsideruar dy njësi leksikore si të përputhshme, duke mundësuar kështu tejkalimin e gabimeve drejtshkrimore, përdorimit të karaktereve speciale dhe ndryshimeve të vogla morfologjike.

Pas përfundimit të fazës së përputhjes, algoritmi hyn në fazën e rekonstruktimit, e cila përcakton strategjitë për kombinimin e informacionit nga të dy dataset-et. Për përputhjet një-me-një, procesi është trivial: fjala ruhet e pandryshuar, duke i bashkëngjitur etiketën NER dhe kategorinë gramatikore nga POS. Ndërsa për përputhjet një-me-shumë, algoritmi aplikon rregulla të qarta: forma finale e fjalës krijohet përmes bashkimit të token-

ëve të POS, ndërsa atributet gramatikore merren nga token-i i parë i grupit, i cili ruan zakonisht funksionin sintaksor më përfaqësues.

Kësisoj, algoritmi siguron ndërtimin e një dataset-i të unifikuar dhe koherent, ku informacioni semantik nga NER kombinohet në mënyrë optimale me informacionin sintaksor dhe gramatikor nga POS, duke garantuar integrim robust edhe në prani të variacioneve në tokenizim dhe drejtshkrim.

3.3.3 Optimizimi i Dataset-it Final

Procesi i optimizimit të dataset-it final konsiston në organizimin e informacionit të kombinuar në një format të standardizuar **CoNLL-U**, i cili përdoret gjerësisht për përfaqësimin e burimeve gjuhësore. Çdo token në dataset-in përfundimtar përbëhet nga dhjetë fusha, duke përfshirë indeksin e fjalës, formën origjinale, lemmën, etiketën gramatikore, tiparet morfologjike, si dhe informacionin mbi marrëdhëniet sintaksore.

Një shtesë e rëndësishme në këtë format është përfshirja e informacionit **NER** në fushën **MISC**, duke krijuar një burim të integruar ku bashkohen të gjitha nivelet e informacionit gjuhësor – morfologjik, sintaksor dhe semantik. Për më tepër, për secilin token llogariten dhe regjistrohen pozicionet e sakta të karaktereve brenda fjalisë, çka mundëson një analizë të detajuar mbi shpërndarjen dhe vendndodhjen e fjalëve në tekst.

Në fazën përfundimtare, të gjitha skedarët individualë në formatin CoNLL-U bashkohen në një dataset unik me strukturë **CSV**. Ky dataset final përmban një organizim më të thjeshtuar me pesë kolona kryesore: **numri i fjalisë, forma e fjalës, lemma, etiketa POS, dhe etiketa NER**.

Dataset-i i optimizuar shërben si një burim i vetëm dhe i qëndrueshëm trajnimit për modelet e mëvonshme, duke siguruar koherencë të plotë ndërmjet burimeve të ndryshme dhe një bazë të fortë për ndërtimin e sistemeve të avancuara të përpunimit të gjuhës natyrore.

KREU IV

ALGORITMAT E PËRDORUR

4.1. Algoritmat Machine Learning

Në këtë punim janë përdorur disa nga algoritmat klasikë dhe të avancuar të cilët janë të dokumentuar në [1]. Eksperimentet janë zhvilluar mbi dataset-in e përmirësuar, duke u fokusuar në vlerësimin e ndikimit që kanë **POS_TAG** dhe **LEMMA** në saktësinë e klasifikimit të **NER_TAG**.

Algoritmat e përdorur përfshijnë:

- **Naive Bayes (NB)** – një klasifikues probabilistik i bazuar në Teoremën e Bayes-it, i cili supozon pavarësinë kushtëzuese të tipareve.
- **Extra Trees (ET)** – algoritëm që ndërton shumë pemë vendimmarrjeje të rastësishme, duke rritur diversitetin e modelit dhe duke ulur variancën.
- **Random Forest (RF)** – një tjetër metodë e bazuar në ndërtimin e një koleksioni pemësh vendimmarrjeje të trajnuara mbi nënbashkësi të rastësishme të të dhënave dhe tipareve.
- **Gradient Boosting (GB)** – teknikë ku modelet e dobëta (zakonisht pemë vendimmarrjeje të cekëta) ndërtohen në mënyrë sekuenciale, duke korrigjuar gabimet e modeleve paraardhëse.
- **Extreme Gradient Boosting (XGBoost)** – një version i optimizuar dhe më efikas i Gradient Boosting, i cili përdor teknika të avancuara për rritjen e shpejtësisë dhe përmirësimin e performancës.
- **Multi-Layer Perceptron (MLP)** – një rrjet artificial nervor i përbërë nga disa shtresa të ndërmjetme, i aftë të mësojë marrëdhënie jolineare komplekse midis tipareve dhe etiketave.

Krahasimi ndërmjet këtyre algoritmave mundëson një kuptim të qartë mbi rolin e tipareve shtesë (**POS_TAG** dhe **LEMMA**) në përmirësimin e performancës së klasifikimit të entiteteve të emëruara.

4.2. BiLSTM

BiLSTM [10] është një arkitekturë e rrjeteve neurale përsëritëse (RNN) që kombinon dy shtresa të veçanta LSTM – njëra që përpunon sekuencën nga e majta në të djathtë dhe tjetra nga e djathta në të majtë [3]. Kjo natyrë dykahëshe i mundëson modelit të kapë informacion kontekstual nga të dyja drejtimet, duke tejkaluar kufizimin e LSTM-ve njëkahëshe që kanë akses vetëm në kontekstin paraprak.

Arkitektura funksionon duke përpunuar input-in në dy rrjedha të ndara, ku çdo fjalë paraqitet në funksion të fjalëve para dhe pas saj. Output-et e dy rrjeteve kombinohen për të ndërtuar një paraqitje më të pasur për çdo token. Ky mekanizëm është veçanërisht i rëndësishëm në detyra të NLP, ku kuptimi i një fjale shpesh varet nga konteksti i plotë i fjalisë. Figura 3.1 paraqet shtresën BiLSTM dhe kahet:

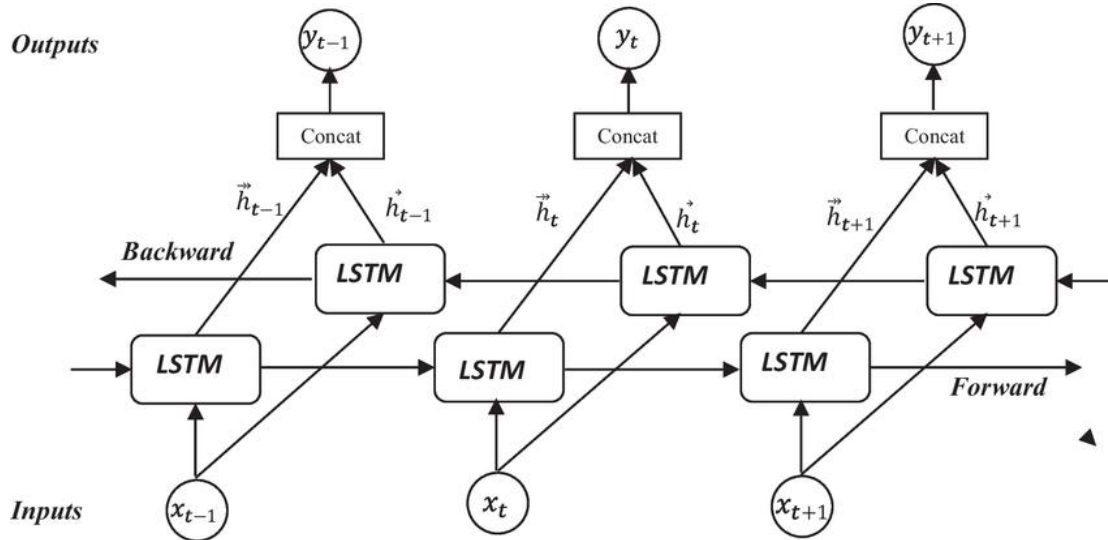


Figura 4.1. Paraqitje e BiLSTM

- **Ndikimi në NER:** BiLSTM është efektiv në identifikimin e entiteteve sepse mund të shfrytëzojë informacionin e plotë të sekuençës. Memoria afatgjatë e LSTM-ve mundëson ruajtjen e informacionit të fjalëve hyrëse gjatë procesimit të entiteteve shumë-fjalëshe, gjë që e bën të përshtatshëm për klasifikimin e saktë të NER_TAG.
- **Ndikimi në POS:** Në detyrën e POS tagging, BiLSTM kontribuon në zgjidhjen e ambiguitetit leksikor duke marrë parasysh strukturën sintaksore të plotë. Për shembull, fjala “drita” mund të jetë **NOUN** (në “drita e diellit”) ose **PROPN** (në “Drita lexon librin”), dhe BiLSTM shfrytëzon kontekstin për të përcaktuar kategorinë e duhur.

4.3. BiLSTM-CRF

Megjithëse BiLSTM ofron një përfaqësim të fuqishëm kontekstual, output-i i tij trajton çdo token në mënyrë të pavarur, pa marrë parasysh varësitë midis etiketave të fjalëve. Për të adresuar këtë kufizim, mbi shtresën e BiLSTM shtohet një **Conditional Random Field (CRF)**, i cili modelon varësitë strukturore midis etiketave dhe siguron një përputhje globale të sekuençës.

CRF është një model probabilistik diskriminues që optimizon të gjithë sekuençën e etiketave duke marrë parasysh tranzicionet midis tyre. Kjo është veçanërisht e dobishme në detyra si NER, ku etiketat kanë lidhje të forta: për shembull, një etiketë B-PER nuk mund të pasohet drejtpërdrejt nga një **I-SHESH** por vetëm nga një **I-PER** ose një etiketë tjetër e lejueshme si **O**.

- **Avantazhet në NER:** Kombinimi BiLSTM-CRF siguron që etiketimi i entiteteve të jetë koherent në nivel sekuece, duke përmirësuar ndjeshëm performancën krahasuar me përdorimin e vetëm BiLSTM. Kjo arkitekturë është bërë standard për NER në shumë gjuhë natyrore.
- **Avantazhet në POS:** Për POS, shtresa CRF ndihmon në ruajtjen e koherencës së strukturës morfosintaksore të fjalive. Për shembull, reduktohet probabiliteti i gjenerimit të etiketave të pamundura në kontekste të caktuara, duke ofruar një model më të qëndrueshëm.

Në përfundim, BiLSTM-CRF [11] përfiton nga fuqia kontekstuale e BiLSTM dhe konsistenca globale e CRF, duke u bërë një nga modelet më të suksesshme dhe të përdorura për detyra të etiketimit sekuencial si NER dhe POS.

4.4. Modele Shtesë

Në këtë punim janë shtuar dhe dy modele shtesë si: BERT dhe RoBERTa si modele Transformer për të vlerësuar përparësitë e tyre, kundrejt modeleve të tjera.

4.4.1. BERT

BERT është një nga modelet më të avancuara të përpunimit të gjuhës natyrore (NLP), i zhvilluar nga Google AI në vitin 2018. Ai përdor arkitekturën Transformer dhe mekanizmin e self-attention për të kapur varësitë afatgjata në tekst. Ndryshe nga modelet tradicionale të gjuhës, BERT analizon kontekstin nga të dyja anët e një fjale, duke mundësuar një kuptim më të thellë dhe më të saktë të kuptimeve të fjalëve në varësi të kontekstit [6].

Trajnimi i BERT bazohet në dy detyra kryesore: Masked Language Modeling (MLM) dhe Next Sentence Prediction (NSP). Në detyrën MLM, një përqindje e fjalëve në një sekuencë janë maskuar dhe modeli duhet të parashikojë fjalët origjinale bazuar në kontekstin përreth, duke ndihmuar në zhvillimin e një kuptimi të thellë të varësive semantike dhe sintaksore brenda një fjale dhe fjalisë [4]. Në detyrën NSP, modeli merr dy fjali dhe duhet të përcaktojë nëse fjalja e dytë është pasuese logjike e së parës, duke ndihmuar në kuptimin e marrëdhënieve midis fjalive dhe strukturës së tekstit [4].

Mekanizmi i self-attention lejon që çdo fjalë në një sekuencë të marrë informacion nga të gjitha fjalët e tjera, duke krijuar prezantime të pasura dhe kontekstuale. Ky mekanizëm është thelbësor për kapjen e varësive komplekse në tekst dhe është një nga arsyt kryesore pse BERT ka performancë të lartë në detyra të ndryshme NLP [4].

BERT është i parapërgatitur në korpuse të mëdha të tekstit, si Wikipedia dhe BooksCorpus, dhe më pas mund të rafinohet (fine-tune) për detyra specifike, duke përfshirë klasifikim teksti, pyetje-përgjigje, analizë sentimentit dhe përpunim të entiteteve të emëruara (NER) [6]. Modeli krijon prezantime kontekstuale të fjalëve, ku e njëjta fjalë

mund të ketë kuptime të ndryshme në kontekste të ndryshme, duke e bërë të veçantë aftësinë e tij për të kuptuar nuancat e gjuhës dhe strukturën sintaksore.

BERT ka një strukturë shumë-shtresore (multi-layered) të encoderëve Transformer, ku çdo shtresë përpunon tekstin në mënyrë hierarkike. Shtresat e thella lejojnë modelin të kapë marrëdhënie komplekse midis fjalëve dhe të ndërtojë prezantime të pasura semantike dhe sintaksore. Aftësia për të rafinuar këtë përfaqësim për detyra specifike e bën BERT shumë fleksibël dhe efektiv.

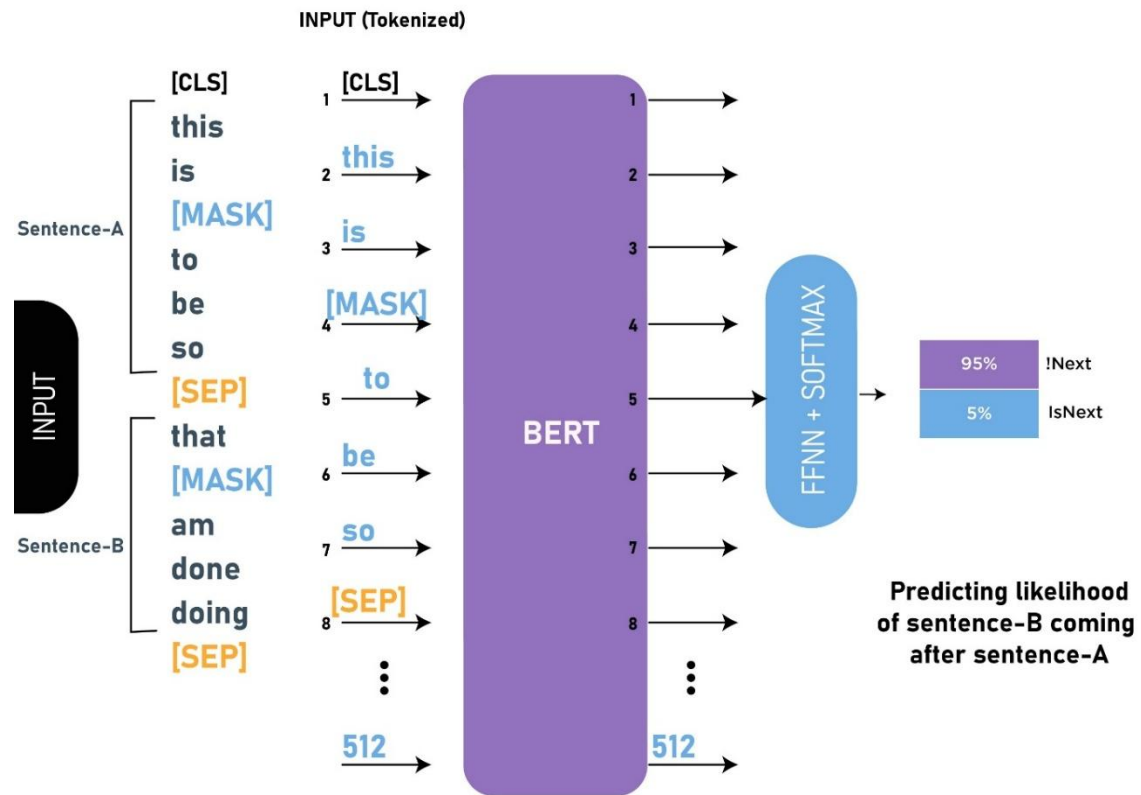


Figura 4.2. Model i BERT

Figura 3.2 ilustron arkitekturën e BERT-it dhe rrjedhën e përpunimit të tekstit brenda modelit. Teksti i hyrjes ndahet në tokena dhe secili token përfaqësohet nga një embedding që kombinon informacionin mbi fjalën, pozicionin e saj dhe segmentin e përkatësisë. Këto embeddings dërgohen në shtresat encoder të Transformer-it, ku mekanizmi i self-attention lejon që çdo token të marrë informacion nga të gjithë tokenat e tjerë në sekuencë. Përmes shtresave të shumta, modeli ndërton prezantime të thelluara dhe kontekstuale për çdo token. Në fund, këto prezantime mund të përdoren për detyra të ndryshme NLP, si NER, POS tagging, pyetje-përgjigje ose klasifikim teksti. Diagrami tregon gjithashtu lidhjen midis tokenëve të hyrjes, shtresave të attention-it dhe prezantimeve finale që përdoren për inferencë të saktë në detyra specifike.

Në detyra si NER, BERT mund të identifikojë entitetet duke përdorur prezantimet kontekstuale dhe mekanizmin e attention për të fokusuar pjesët më të rëndësishme të tekstit. Ai kap marrëdhëniet semantike midis fjalëve që përbëjnë entitete komplekse, duke përmirësuar saktësinë në identifikimin e tyre. Në detyra POS, BERT dallon kategoritë gramatikore të fjalëve dhe kuptimin e tyre sintaksor, duke marrë parasysh përdorimet e ndryshme të së njëjtës fjalë në kontekste të ndryshme gramatikore.

Përveç NER dhe POS, BERT përdoret gjerësisht edhe në pyetje-përgjigje, ku ndihmon në gjetjen e përgjigjeve të sakta nga tekste të mëdha, si dhe në analizë sentimentit, ku identifikon emocionet dhe opinionet e shprehura në tekst. Modeli përdoret gjithashtu për përkthime automatike, përmbledhje të teksteve dhe klasifikim tematik, duke bërë që të jetë një mjet i fuqishëm për shumë aplikacione të NLP.

Një nga avantazhet kryesore të BERT është aftësia për të krijuar prezantime të pasura dhe kontekstuale që mund të transferohen nga një detyrë në një tjetër, gjë që zvogëlon nevojën për veçori manuale dhe përpunim të shtuar të të dhënave. Megjithatë, trajnimi dhe inferenca kërkojnë burime të mëdha llogaritëse, përfshirë GPU ose TPU, gjë që mund të kufizojë përdorimin në sisteme me kapacitete të ulëta.

BERT ka shënuar një ndryshim të madh në fushën e NLP dhe ka shërbyer si bazë për shumë modele të tjera moderne, përfshirë RoBERTa, ALBERT dhe DistilBERT, të cilat e përmirësojnë performancën, reduktojnë kompleksitetin ose optimizojnë kohën e trajnimit dhe inferencës. Për shkak të këtyre karakteristikave, BERT vazhdon të përdoret gjerësisht në kërkimin akademik, aplikacione industriale dhe sisteme të inteligjencës artificiale që përpunojnë gjuhën natyrore.

4.4.2. RoBERTa

RoBERTa është një version i përmirësuar dhe i optimizuar i BERT që synon të përmirësojë performancën në detyra të ndryshme të përpunimit të gjuhës natyrore (NLP) [5]. Ndryshe nga BERT, RoBERTa eliminon detyrën e Next Sentence Prediction (NSP) dhe përqendrohet plotësisht në detyrën e Masked Language Modeling (MLM), duke lejuar që modeli të mësojë një kuptim më të saktë të kontekstit të tekstit. Ky fokus në MLM i mundëson modelit të zhvillojë prezantime më të forta dhe të përgjithësueshme, duke përmirësuar aftësinë për të kapur marrëdhënie semantike dhe sintaksore brenda sekuencave të gjata teksti [6].

Trajnimi i RoBERTa përdor batch-e më të mëdha, periudha më të gjata trajnimi dhe sekuenca më të gjata teksti në krahasim me BERT-in. Kjo i jep modelit mundësinë të mësojë varësi më komplekse midis fjalëve dhe të ndërtojë prezantime kontekstuale më të thelluara. Një nga inovacionet kryesore të RoBERTa është përdorimi i maskimit dinamik, ku pozicionet e fjalëve të maskuara ndryshojnë në çdo epokë trajnimi. Kjo qasje rrit variacionin e të dhënave të trajnimit, përmirëson përgjithësueshmërinë e modelit dhe e bën më të qëndrueshëm në trajtimin e teksteve që nuk janë parë më parë [6].

Maskimi dinamik ndryshon pozicionet e fjalëve të fshehura për të njëjtën fjali në epoka të ndryshme, duke krijuar një para-trajnim më të fortë dhe duke siguruar që modelit nuk i mungojnë variacione të rëndësishme kontekstuale. Eliminimi i detyrës NSP lejon që modeli të përqendrohet në kuptimin e lidhjeve midis fjalëve brenda fjalisë dhe të përmirësojë aftësinë për të kuptuar kontekstin lokal dhe global [6].

RoBERTa përdor një arkitekturë shumë-shtresore të encoderëve Transformer, ku çdo shtresë përdor mekanizmin e self-attention për të kapur lidhjet midis të gjitha tokenëve të një sekuençe. Ky mekanizëm i attention-it lejon që secili token të marrë informacion nga çdo token tjetër, duke ndihmuar në krijimin e prezantimeve të pasura dhe kontekstuale që reflektojnë marrëdhëniet komplekse midis fjalëve. Trajnimi i zgjeruar dhe përdorimi i sekuençave më të gjata e bëjnë RoBERTa veçanërisht të aftë për të kuptuar struktura komplekse morfosintaksore dhe semantike në tekst [6].

Në detyra të përpunimit të entiteteve të emëruara (NER), RoBERTa shfaq performancë më të mirë krahasuar me BERT-in e zakonshëm. Strategjitë e përmirësuara të trajnimit dhe maskimi dinamik ndihmojnë modelin të mësojë prezantime më të forta dhe të përgjithësojë më mirë për entitete të reja që nuk janë parë gjatë trajnimit [7]. Ai është veçanërisht efektiv në kapjen e marrëdhënieve semantike midis fjalëve që përbëjnë entitete komplekse, duke e bërë modelin të përshtatshëm për detyra të sfiduara si identifikimi i organizatave, vendeve, datave dhe emrave të personave.

Në detyra të klasifikimit të pjesëve të fjalës (POS tagging), RoBERTa ka aftësi të përmirësuara për të kuptuar nuancat morfosintaksore. Trajnimi i zgjatur, përdorimi i batch-eve më të mëdha dhe sekuençave më të gjata i mundësojnë modelit të dallojë përdorime të ndryshme të së njëjtës fjalë në kontekste të ndryshme dhe të njohë marrëdhënie më komplekse midis fjalëve dhe kategorive gramatikore. Kjo e bën RoBERTa shumë të dobishëm në detyra ku preciziteti gramatikor dhe kuptimi kontekstual janë të rëndësishme [7].

Përveç NER dhe POS, RoBERTa përdoret gjerësisht në pyetje-përgjigje, klasifikim teksti, analizë sentiment, përmbledhje të teksteve dhe përkthime automatike. Strategjitë e përmirësuara të trajnimit dhe maskimi dinamik e bëjnë modelin të përgjithësueshëm dhe të aftë për të trajtuar të dhëna të reja me saktësi të lartë. Për shkak të këtyre karakteristikave, RoBERTa është një nga modelet më të fuqishme dhe të përdorura gjerësisht në NLP [6].

Avantazhet kryesore të RoBERTa qëndrojnë në aftësinë për të krijuar prezantime të pasura dhe kontekstuale, performancën e lartë në detyra të ndryshme NLP, përgjithësueshmërinë e madhe për të trajtuar tekste të reja dhe aftësinë për të përmirësuar performancën krahasuar me BERT-in e zakonshëm. Megjithatë, trajnimi dhe inferenca e tij kërkojnë burime të mëdha llogaritëse, duke përfshirë GPU ose TPU, gjë që mund të kufizojë përdorimin në sisteme me kapacitete të ulëta [6].

KREU V

EKSPERIMENTET

5.1. Mënyra e eksperimentit të modeleve të thjeshta

Eksperimenti u zhvillua duke ndjekur një proces të strukturuar që teston 8 algoritme të ndryshme në dy mënyra.

5.1.1. Struktura e programit

Programi është strukturuar në mënyrë të qartë dhe funksionale në tre pjesë kryesore që sigurojnë një rrjedhë të organizuar të punës dhe një analizë të plotë të të dhënave për NER.

Së pari, ngarkimi i të dhënave realizohet nga një skedar CSV i cili përmban informacion të pasur në forma të ndryshme: fjalët (WORD), etiketat për NER (NER_TAG), etiketat gramatikore të pjesës së fjalës (POS_TAG), dhe temat ose format bazë të fjalëve (LEMMA). Kjo strukturë mundëson që çdo njësi gjuhësore të trajtohet në mënyrë të detajuar dhe të përgatitet për modelim të mëtejshëm.

Së dyti, përpunimi i veçorive realizohet në dy qasje kryesore: versioni bazë fokusohet thjesht tek fjalët dhe i konverton ato në numra përmes kodimit të etiketave, duke krijuar një përfaqësim minimal dhe të drejtpërdrejtë të informacionit; ndërsa versioni i përmirësuar kombinon fjalët me etiketat POS dhe temat, duke gjeneruar një set veçorish më kompleks dhe më informativ që lejon algoritmet të kapin marrëdhënie më të thella morfologjike dhe kontekstuale.

Së fundi, testi sistematik i algoritmeve aplikohet mbi të dyja variantet e veçorive për të vlerësuar performancën dhe për të krahasuar ndikimin e shtimit të informacionit kontekstual. Kjo qasje e organizuar siguron një eksperiencë të plotë eksperimentale, duke lejuar që rezultatet të jenë të krahasueshme, të interpretuara lehtësisht dhe të shërbejnë si bazë për përmirësime të mëtejshme në modele të ndryshme të NLP-së.

5.1.2. Metrikat

F1 score është një mënyrë për të matur sa mirë një model po identifikon gjëra të veçanta, duke kombinuar precizionin (saktësinë e parashikimeve pozitive) dhe recall (sa shumë nga të vërtetat pozitive modeli ka kapur). Për krahasimin e modeleve përdoret Token F1 dhe Token Entity F1.

- Token F1- tregon sa mirë modeli etiketon token-at individualë. Është më tolerant.
- Entity F1- tregon sa mirë modeli gjen entitetet e plota dhe tipin e tyre të saktë. Është më i rreptë dhe më afër përdorimit real.

Në klasifikimin NER, përgjithësisht dataset-i dominohet nga tagu O. Kjo bën që modelet klasifikuese të parashikojnë këtë tag me shumë. Kjo sjell saktësi dhe Token F1 të lartë, por Entity F1 tregon me mirë performancën e modelit me entitetet dhe a është në gjendje t'i parashikojë ato.

5.1.3. Libraritë dhe Importet

Sistemi përdor një kombinim librarish për implementimin e plotë të zgjidhjes së NER-it. Nga *sklearn* importohen algoritmet kryesore si *MultinomialNB*, *RandomForestClassifier*, *ExtraTreesClassifier*, *GradientBoostingClassifier* dhe *MLPClassifier*, së bashku me mjetet për ndarjen e të dhënave *train_test_split* dhe vlerësimin *f1_score*. Libraria *LabelEncoder* përdoret për konvertimin e të dhënave kategorike në numra. *XGBoost* importohet për gradient boosting të optimizuar, ndërsa *seqeval.metrics* ofron metrikat specifike për vlerësimin e sekuencave NER. *Pandas* menaxhon të dhënat, *numpy* kryen operacionet numerike, dhe *time* mat kohën e trajnimit.

5.1.4. Procesi i Përpunimit të të Dhënave

Pas ngarkimit të datasetit nga CSV, sistemi implementon dy klasa kryesore. Klasa e parë *MachineLearningAlgorithmsNER* punon vetëm me fjalët si veçori, ndërsa *MachineLearningAlgorithmsNERFeatures* përdor një kombinim fjalësh, etiketash POS dhe temash. Në të dyja rastet, procesi fillon me metodën *get_word_and_tags_2_ids* që ekstraktin kolonat përkatëse nga dataseti dhe krijon enkodera për çdo tip të dhëne. *LabelEncoder* transformon çdo fjalë, etiketë POS, temë dhe etiketë NER në një indeks numerik. Sistemi krijon harta *word2idx/idx2word* dhe *tag2idx/idx2tag* për konvertim të lehtë midis formave tekstuale dhe numerike.

5.1.5. Strukturimi i të Dhënave për Trajnim

Për modelet që përdorin vetëm fjalët, *self.X* formohet si një vektor kolonë i indekseve të fjalëve duke përdorur *reshape(-1, 1)*. Për modelet me veçori shtesë, tre vektorë të veçorive kombinohen duke përdorur *np.hstack* për të krijuar një matricë ku çdo rresht përfaqëson një token me tre dimensione. *self.Y* përfaqëson etiketat e synuara NER të koduara numerikisht. Metoda *split_dataset* ndan të dhënat në 80% trajnim dhe 20% testim duke përdorur *train_test_split*.

5.1.6. Procesi i Trajnimit dhe Vlerësimit

Çdo algoritëm implementohet nëpërmjet metodës *_train_and_evaluate* që mat kohën e trajnimit, trajnon modelin me *model.fit(X_train, y_train)*, dhe bën parashikime me *model.predict(X_test)*. Vlerësimi bëhet me dy metrika. Token F1 llogaritet duke përdorur *fl_score* me *average='micro'* që mat saktësinë në nivel individual të tokenëve. Entity F1 përdor *segeval* që vlerëson performancën në nivel entiteti duke marrë parasysh strukturën BIO të etiketimit.

```
def _train_and_evaluate(self, model, X_train, y_train, X_test, y_test):
    start_time = time.time()
    model.fit(X_train, y_train)
    training_time = time.time() - start_time
    y_pred = model.predict(X_test)
    token_f1 = fl_score(y_test, y_pred, average='micro')
    y_true_tags = [self.idx2tag[idx] for idx in y_test]
    y_pred_tags = [self.idx2tag[idx] for idx in y_pred]
    entity_f1 = entity_f1_score([y_true_tags], [y_pred_tags])
    self.model = model
    print(f'Finished Training: {self.model_name}')
    return {
        'Model': self.model_name,
        'Token F1': token_f1,
        'Entity F1': entity_f1,
        'Training Time': training_time
    }
```

5.1.7. Implementimi i MPL

Klasat *MLPNER* dhe *MLPNERFeatures* implementojnë rrjete neurale shumëshresore. Këto klasa ruajnë ID-të e fjalive *sentence_ids* për të mundësuar vlerësimin në nivel fjalie. Metoda *convert_to_segeval_format* organizon parashikimet dhe vlerat e vërteta sipas fjalive, duke krijuar lista të etiketave për çdo fjalë. Kjo është e nevojshme sepse *segeval* kërkon strukturën e fjalive për të vlerësuar entitetet në mënyrë të saktë. Sistemi ofron tre konfigurime të MLP me numër të ndryshëm shtresash dhe neuronash, të gjitha duke përdorur funksionin e aktivizimit *ReLU* dhe optimizuesin *Adam*.


```

def train(self, classifier="mlps"):
    classifiers = {
        "mlps": MLPClassifier(hidden_layer_sizes=(40,), max_iter=250,
activation="relu", solver="adam"),
        "mlpm": MLPClassifier(hidden_layer_sizes=(120, 40), max_iter=250,
activation="relu", solver="adam"),
        "mlpl": MLPClassifier(hidden_layer_sizes=(100, 120, 40), max_iter=250,
activation="relu", solver="adam"),
    }
    model = classifiers[classifier]
    start_time = time.time()
    model.fit(self.X_train, self.Y_train)
    training_time = time.time() - start_time
    y_pred = model.predict(self.X_test)
    token_f1 = f1_score(self.Y_test, y_pred, average="micro")
    y_true_seqeval, y_pred_seqeval = self.convert_to_seqeval_format(self.Y_test,
y_pred, self.sent_ids_test)
    entity_f1 = entity_f1_score(y_true_seqeval, y_pred_seqeval)
    print(f'Finished Training: {self.model_name}')
    return {
        'Model': self.model_name,
        'Token F1': token_f1,
        'Entity F1': entity_f1,
        'Training Time': training_time
    }

```

5.1.8. Ekzekutimi dhe Mbledhja e Rezultateve

Në fund, sistemi ekzekuton të gjithë algoritmet në mënyrë sekuenciale duke thirrur funksionet përkatëse për çdo model. Çdo funksion krijon një instancë të klasës së duhur, inicializon koduesit, ndan të dhënat, trajnon modelin dhe kthen rezultatet në formë fjalorth që përmban emrin e modelit, Token F1, Entity F1 dhe kohën e trajnimit. Të gjitha rezultatet

mbliidhen në një listë dhe konvertohen në një DataFrame për analizë dhe krahasim të lehtë të performancës së modeleve të ndryshme.

5.2. Mënyra e eksperimentit të BiLSTM-ve

5.2.1. Libraritë dhe Importet

Sistemi përdor një përzierje librarish për implementimin e modeleve. Nga *PyTorch* importohen komponente kryesore si *torch.nn* për ndërtimin e rrjeteve neurale, *DataLoader* për menaxhimin e të dhënave në batch, dhe optimizuesit për trajnimin. *torchcrf* ofron implementimin e CRF për sekuencat, ndërsa *segeval* siguron metrikat specifike për vlerësimin e NER-it. *Tensorflow* përdoret vetëm për funksionin *pad_sequences* që uniformizon gjatësinë e sekuencave. *sklearn* kontribuon me mjetet për ndarjen e të dhënave dhe matjen e performancës.

5.2.2. Procesi i Përpunimit të të Dhënave

Klasa *NERDataProcessor* menaxhon të gjithë procesin e parapërgatitjes. Fillimisht, të dhënat ngarkohen nga CSV dhe grupohen sipas fjalive duke përdorur *groupby("SENTENCE #")*. Për çdo tip të dhëne krijohen fjalorë të specializuar. Për fjalët dhe temat, aplikohet *.str.lower()* para krijimit të setit unik për të standardizuar formën. Çdo fjalor përfshin tokenat speciale **<PAD>** për padding dhe **<UNK>** për fjalë të panjohura, me përjashtim të etiketave NER që kërkojnë vetëm **<PAD>**. Gjatësia maksimale e sekuencave përcaktohet 128. Sekuencat më të shkurtra se kjo gjatësi plotësohen me padding në fund duke përdorur *pad_sequences*.

5.2.3. Arkitekturat e Modeleve

Sistemi implementon gjashtë variante të ndryshme të modeleve që ndahen në dy grupe kryesore.

5.2.3.1. BiLSTM i Thjeshtë

Arkitektura e modelit të parë përbën bazën e të gjithë eksperimenteve të tjera dhe implementon një model minimale por efektiv për NER. Ky model përdor vetëm informacionin e fjalëve si hyrje, duke mos integruar veçori shtesë. Struktura kryesore fillon me një shtresë *embedding* që transformon çdo fjalë në një vektor 200-dimensional, e cila mundëson modelit të mësojë përfaqësimet semantike të fjalëve gjatë procesit të trajnimit.

Këto vektorë më pas kalojnë nëpër një rrjet LSTM dykahësh me dy shtresa, ku çdo shtresë ka 128 njësi të fshehura në secilën drejtim, duke krijuar një kapacitet total prej 256 njësish për çdo shtresë.

Komponenti BiLSTM është thelbësor për performancën e modelit sepse mundëson procesuarin e informacionit kontekstual nga të dyja drejtimet e sekuencës së fjalëve. Kjo karakteristikë është veçanërisht e rëndësishme për NER sepse identifikimi i saktë i entiteteve shpesh varet nga konteksti i plotë rreth tyre, jo vetëm nga fjalët që i paraprijnë. Shtresa dropout me vlerë 0.3 aplikohet pas BiLSTM-it për të reduktuar rrezikun e overfitting-ut, ndërsa klasifikuesi linear final konverton daljet 256-dimensionale të BiLSTM-it në probabilitete për çdo etiketë të mundshme NER. Figura 5.1 paraqet arkitekturën e modelit:

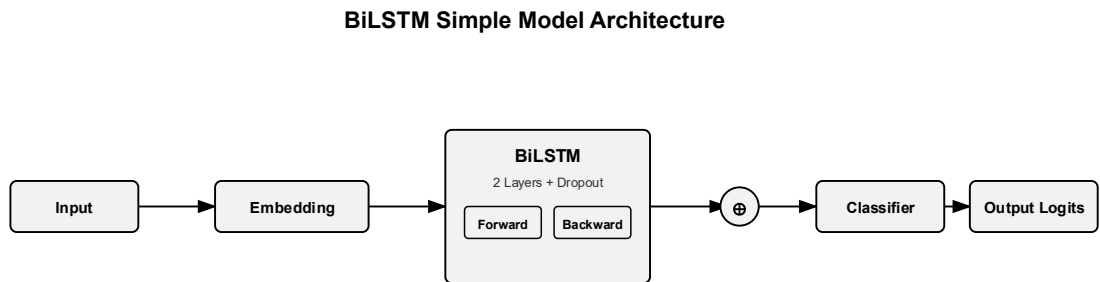


Figura 5.1. Arkitektura e BiLSTM të Thjeshtë

Aspekti i trajnimit të këtij modeli integron disa teknika të rëndësishme optimizimi. Optimizuesi Adam përdoret me një shkallë mësimi prej 0.001, duke ofruar një balancë të mirë midis shpejtësisë së konvergjencës dhe stabilitetit të trajnimit. Gradient clipping me një normë maksimale prej 1.0 aplikohet për të parandaluar shpërthimin e gradientëve, një problem i zakonshëm në RNN. Funkcioni i humbjes *CrossEntropyLoss* përdoret duke injoruar token-at e padding-ut, duke siguruar që vetëm token-at aktualë të kontribuojnë në procesin e mësimi.

5.2.3.2. BiLSTM me Veçori POS

Modeli i dytë zgjeron arkitekturën bazike duke integruar informacionin POS, duke krijuar një sistem më të sofistikuar që shfrytëzon njohjen sintaksore të gjuhës. Arkitektura e re përdor dy shtresa embedding të ndara dhe të specializuara: një për fjalët me 200 dimensione dhe një tjetër për etiketat POS me 50 dimensione. Kjo ndarje lejon modelin të mësojë përfaqësime të pavarura për informacionin sintaksor, duke optimizuar kapacitetin e mësimi për të dyja llojet e informacionit. Kombinimi i këtyre vektorëve bëhet përmes bashkimit, duke krijuar një vektor hyrës 250-dimensionale që pasqyron si përmbajtjen semantike ashtu edhe rolin gramatikor të çdo fjale. Figura 5.2. paraqet arkitekturën e modelit:

BiLSTM with POS Features Model Architecture

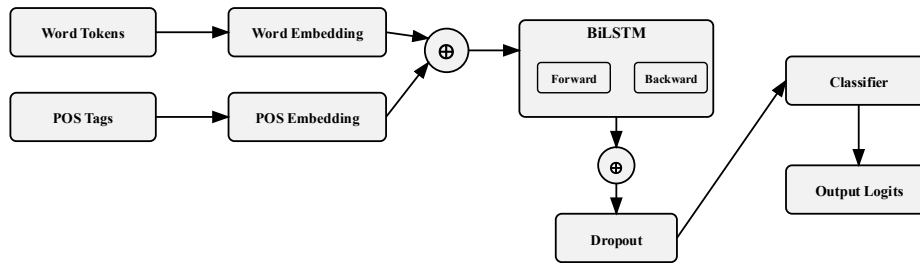


Figura 5.2. Arkitektura e BiLSTM me POS

Integrimi i etiketave POS sjell përfitime të konsiderueshme për detektimin e entiteteve sepse shumë kategori entitetesh kanë modele sintaksore të parashikueshme.

5.2.3.3. BiLSTM me Veçori të Plota

Modeli i tretë paraqet implementimin më të plotë të veçorive gjuhësore, duke integruar fjalët, temat dhe etiketat POS në një arkitekturë të unifikuar që synon të shfrytëzojë maksimalisht informacionin e disponueshëm nga teksti. Arkitektura përdor tre shtresa embedding të specializuara: fjalët me 200 dimensione, lemat me 50 dimensione, dhe etiketat POS me 50 dimensione, duke krijuar një hyrje të kombinuar 300-dimensionale. Lemat ofrojnë një shtresë shtesë abstraksioni duke reduktuar fjalët në format e tyre bazë, duke lejuar modelin të identifikojë lidhje semantike midis formave të ndryshme të së njëjtës fjalë. Kjo është veçanërisht e vlefshme për gjuhë me morfologji të pasur ku një fjalë mund të ketë shumë forma sipërfaqësore. Figura 5.3. paraqet arkitekturën e modelit:

BiLSTM with Multiple Features Model Architecture

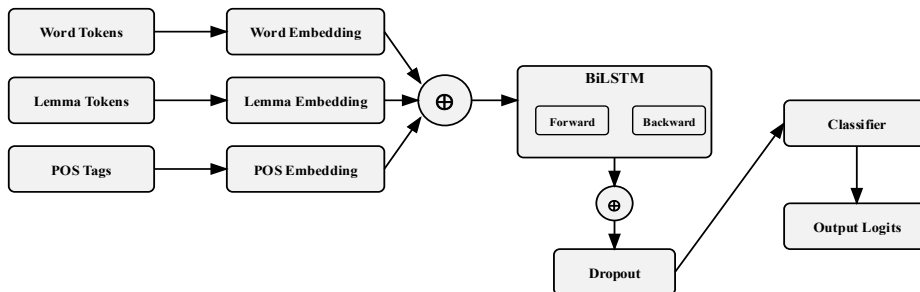


Figura 5.3. Arkitektura e BiLSTM (LEMMA + POS)

Integrimi i LEMMA krijon një dimension të ri analitik për modelin, duke i mundësuar atij të kapë modele që mund të ishin të fshehura në variantet morfologjike të fjalëve. Për shembull, emrat e vendeve mund të shfaqen në forma të ndryshme por LEMMA i redukton të gjitha këto në formën bazë, duke lehtësuar identifikimin e entitetit.

Kombinimi i tre llojeve të informacionit krijon një përfaqësim shumë-dimensional që kap aspekte semantike, morfologjike dhe sintaksore të tekstit. BiLSTM-i duhet të përpunojë tani një sasi më të madhe informacioni, por edhe ka akses në një spektër më të gjerë njohurish linguistike për të marrë vendime klasifikimi.

5.2.3.4. BiLSTM-CRF i Thjeshtë

Modeli i katërt sjell një ndryshim në arkitekturë duke zëvendësuar shtresën e klasifikimit linear me një CRF, duke krijuar një qasje hibride që kombinon fuqinë e BiLSTM-it për nxjerrjen e veçorive me aftësinë e CRF-së për modelimin e sekuencave. Ndërsa pjesa BiLSTM mbetet identike me modelin e parë, shtimi i CRF-së transformon modelin nga një klasifikues i pavarur token-ash në një optimizues sekuence që merr parasysh varësitë midis etiketave fqinje. CRF-ja modelon shpërndarjen e kushtëzuar të gjithë sekuencës së etiketave, duke mundësuar aplikimin e kufizimeve strukturore që ekzistojnë natyrisht në skemën e etiketimit BIO. Figura 5.4. paraqet arkitekturën e modelit:

BiLSTM-CRF Simple Model Architecture

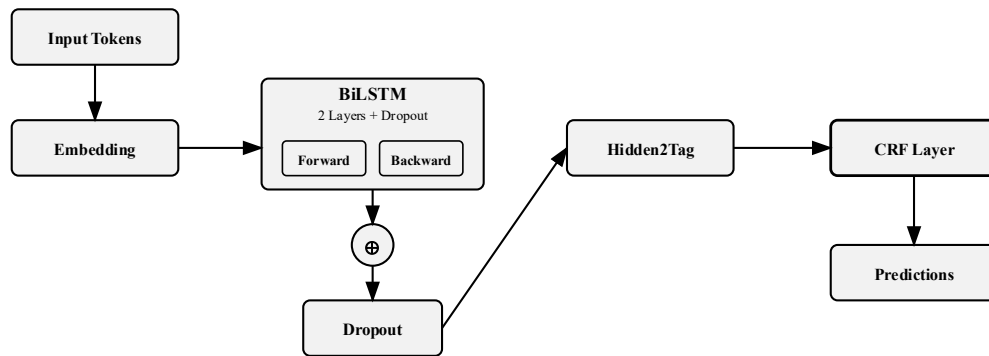


Figura 5.4. Arkitektura e BiLSTM-CRF të Thjeshtë

Avantazhi kryesor i CRF-së qëndron në aftësinë e saj për të parandaluar sekuenca të pavlefshme etiketash, si për shembull kalimin e drejtpërdrejtë nga O në I-PER pa një etiketë B-PER të ndërmjetme. Kjo karakteristikë është thelbësore për NER sepse rregullat e etiketimit BIO kanë kuptim gjuhësor dhe strukturor të rëndësishëm. Ndërsa BiLSTM-i ka tendencën të bëjë gabime lokale në klasifikimin e token-ave individualë, CRF-ja optimizon gjithë sekuencën njëkohësisht, duke rezultuar në parashikime më koherente dhe logjiksht të vlefshme.

5.2.3.5. BiLSTM-CRF me Veçori POS

Modeli i pestë kombinon avantazhet e informacionit sintaksor me fuqinë e modelimit të sekuecave, duke krijuar një arkitekturë që shfrytëzon si veçoritë POS ashtu edhe kufizuesit strukturorë të CRF-së për një performancë të përmirësuar. Arkitektura pasqyron modelin e dytë në pjesën e BiLSTM-it, duke përdorur embedding-e të ndara për fjalët (200 dimensione) dhe etiketat POS (50 dimensione), por zëvendëson klasifikuesin linear me një shtresë CRF për parashikimin final. Kjo kombinim krijon një model që mund të shfrytëzojë informacionin gramatikor për të marrë vendime më të informuara për klasifikimin e entiteteve, ndërsa siguron që parashikimet e etiketave të jenë strukturalisht të vlefshme. Figura 5.5. paraqet arkitekturën e modelit:

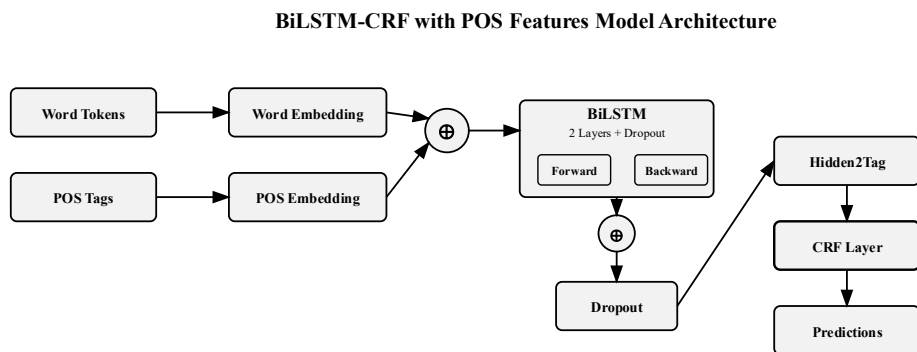


Figura 5.5. Arkitektura e BiLSTM-CRF (POS)

Sinergia midis veçorive POS dhe CRF-së është veçanërisht e fuqishme sepse të dyja kontribuojnë në aspekte të ndryshme të saktësisë së modelit. Etiketat POS ofrojnë informacion sintaksor që ndihmon në identifikimin e kandidatëve potencialë për entitete (p.sh., emrat e përveçëm janë kandidatë të mirë për emra personash), ndërsa CRF-ja siguron që klasifikimet e bëra të jenë koherente përgjatë gjithë sekuecës. Kjo është veçanërisht e vlefshme për entitete shumë-fjalëshe, ku POS tags mund të ndihmojnë në identifikimin e kufijve të entitetit dhe CRF-ja siguron që të gjitha token-at e entitetit të klasifikohen në mënyrë konsistente. Kombinimi rezulton në një model që është më pak i prirur ndaj gabimeve të segmentimit dhe më i saktë në identifikimin e entiteteve komplekse.

5.2.3.6. BiLSTM-CRF me Veçori të Plota

Modeli i fundit paraqet arkitekturën më të sofistikuar dhe të plotë, duke kombinuar të gjitha veçoritë e mundshme gjuhësore me tekniken më të avancuar të modelimit të sekuecave të disponueshme në këtë studim. Arkitektura integron tre lloje embedding-ësh: fjalët (200 dimensione), lemat (50 dimensione), dhe etiketat POS (50 dimensione), duke krijuar një hyrje 300-dimensionale që më pas përpunohet nga BiLSTM-i dhe CRF-ja. Ky model synon të shfrytëzojë plotësisht të gjithë informacionin e disponueshëm ndërsa aplikon teknikat më të mira të modelimit të sekuecave për të arritur

performancën teorikisht më të lartë të mundshme me këto veçori. Figura 5.6. paraqet arkitekturën e modelit:

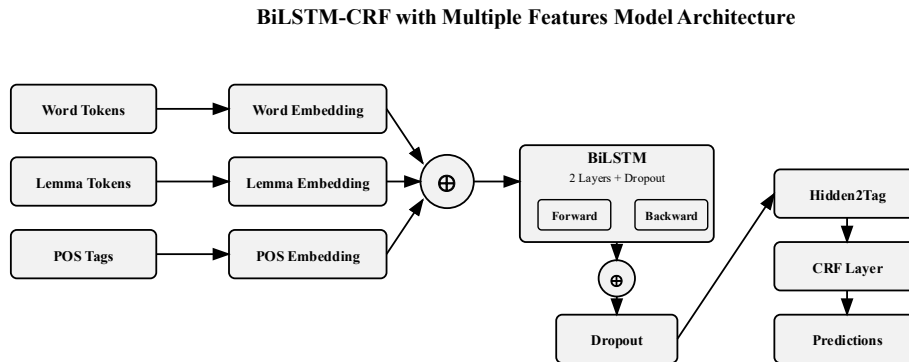


Figura 5.6. Arkitektura e BiLSTM-CRF (POS + LEMMA)

Sinergjia e tre llojeve të veçorive me CRF-në krijon një model shumë të fuqishëm por edhe kompleks. Fjalët ofrojnë informacion leksikor të drejtpërdrejtë, temat kapin modele semantike të përgjithësuara duke abstraguar variantet morfologjike, dhe etiketat POS sigurojnë kontekst sintaksor. CRF-ja koordinon të gjitha këto informacione për të marrë vendime koherente të sekuencës që respektojnë kufizuesit strukturorë të skemës BIO. Ky kombinim është veçanërisht i vlefshëm për entitete komplekse shumë-fjalëshe ku lidhja midis semantikës, morfologjisë dhe sintaksës është kritike për identifikim të saktë. Modeli ka kapacitetin teorik më të lartë për të mësuar dhe parashikuar modele komplekse në të dhënat e NER-it.

Implementimi dhe trajnimi i këtij modeli paraqet sfidat më të mëdha në terma të kompleksitetit llogaritës dhe rrezikut të overfitting-ut. Tre shtresat e embedding-ut rrisin ndjeshëm numrin e parametrave, ndërsa hyrja 300-dimensionale në BiLSTM kërkon më shumë kohë për procesim. CRF-ja duhet të koordinojë informacion nga tre burime të ndryshme, duke bërë procesin e optimizimit më të vështirë dhe më të ngadaltë.

Tabela 5.1. Parametra të modeleve

Modeli	Embeddig Dimension	Loss	CRF
BiLSTM	200	CrossEntropyLoss	No
BiLSTM + POS	250	CrossEntropyLoss	No
BiLSTM Full	300	CrossEntropyLoss	No
BiLSTM-CRF	200	CRF Loss	Yes
BiLSTM-CRF + POS	250	CRF Loss	Yes
BiLSTM-CRF Full	300	CRF Loss	Yes

5.2.4. Menaxhimi i Veçorive

Modelet e thjeshta punojnë vetëm me embedding-un e fjalëve që konvertohen në vektorë 200-dimensionalë. Modelet me POS shtojnë embedding-un e etiketave gramatikore me 50 dimensione. Modelet me veçori të plota kombinojnë tre embedding-e: fjalët (200 dim), lemmat (50 dim), dhe etiketat POS (50 dim). Kombinimi bëhet duke përdorur *torch.cat* në dimensionin e dytë, duke krijuar vektorë input 300-dimensionalë për rrjetin LSTM. Të gjitha embedding-et kanë parametrin *padding_idx* të konfiguruar për të trajtuar në mënyrë korrekte token-at e padding-ut.

```
def forward(self, words, lemmas, pos_tags, tags=None, mask=None):
    word_emb = self.word_embedding(words)
    lemma_emb = self.lemma_embedding(lemmas)
    pos_emb = self.pos_embedding(pos_tags)

    combined_emb = torch.cat([word_emb, lemma_emb, pos_emb], dim=2)
    lstm_out, _ = self.lstm(combined_emb)
    lstm_out = self.dropout(lstm_out)
    logits = self.hidden2tag(lstm_out)

    if tags is not None:
        loss = -self.crf(logits, tags, mask=mask, reduction="mean")
        return {"loss": loss}
    else:
        predictions = self.crf.decode(logits, mask=mask)
        return {"predictions": predictions}
```

5.2.5. Procesi i Trajnimit dhe Early Stopping

Klasa *EarlyStopping* implementon një mekanizëm për ndaljen e hershme të trajnimit. Monitoron performancën e validimit dhe ndal trajnimin nëse nuk ka përmirësim për një numër të caktuar epokash (*patience*=5). Përdor *min_delta*=0.0001 si kufi minimal për të konsideruar një përmirësim të vërtetë. Kur aktivizohet early stopping, modeli rikthehet automatikisht në peshat më të mira të ruajtura gjatë trajnimit. Procesi i trajnimit përfshin *gradient clipping* me *max_norm*=1.0 për të parandaluar gradientët e pakontrollueshëm, dhe përdor optimizer-in *Adam* me *learning rate* 0.001.

```
class EarlyStopping:

    def __init__(self, patience=5, min_delta=0.0001, restore_best_weights=True):
        self.patience = patience
        self.min_delta = min_delta
        self.restore_best_weights = restore_best_weights
```



```

self.best_score = None
self.counter = 0
self.best_weights = None
self.early_stop = False

def __call__(self, val_score, model):
    if self.best_score is None:
        self.best_score = val_score
        self.save_checkpoint(model)
    elif val_score < self.best_score + self.min_delta:
        self.counter += 1
        print(f'EarlyStopping counter: {self.counter} out of {self.patience}')
        if self.counter >= self.patience:
            self.early_stop = True
    else:
        self.best_score = val_score
        self.save_checkpoint(model)
        self.counter = 0

def save_checkpoint(self, model):
    if self.restore_best_weights:
        self.best_weights = model.state_dict().copy()

```

5.2.6. Ekzekutimi dhe Optimizimi

Funksioni kryesor `main()` koordinon të gjithë procesin duke inicializuar processerin e të dhënave, duke ndërtuar fjalorin, dhe duke krijuar ndarjet `train/validation/test` në 80/10/10. Të dhënat konvertohen në tensor dhe transferohen në GPU nëse disponohet. Çdo model trajnohet me `early stopping` deri në maksimum 50 epoka por zakonisht ndalet më herët kur performanca stabilizohet. Rezultatet ruhen në `DataFrame` për analizë dhe krahasim të performancës së modeleve. Kodi i mëposhtëm paraqet këtë kod

5.3. Mënyra e eksperimentit të BERT dhe RoBERTa

5.3.1. Libraritë dhe Importet

Sistemi përdor librari për implementimin e modeleve `transformer`. Nga *transformers* importohen komponentët kryesorë si *AutoTokenizer* dhe *AutoModelForTokenClassification* për ngarkimin automatik të modeleve të paratrajnuara, së bashku me *TrainingArguments* dhe *Trainer* për menaxhimin e trajnimit. *DataCollatorForTokenClassification* siguron batch-imin e saktë të të dhënave për klasifikimin e token-ave, ndërsa *XLNetForTokenClassification* ofron arkitekturën e

specializuar për *RoBERTa*. *datasets.Dataset* përdoret për konvertimin e të dhënave në formatin e kërkuar nga *HuggingFace*, nga ngarkohen dhe modelet.

5.3.2. Parapërgatitja e të Dhënave dhe Hartimi i Etiketave

Të dyja klasat BERT dhe RoBERTa fillojnë procesin duke ekstraktuar etiketa unike nga dataseti dhe duke krijuar hartat *label2id* dhe *id2label* për konvertimin midis etiketave tekstuale dhe indekseve numerike. Metoda *get_sentences()* transformon dataframe-in në lista (fjala, etiketa) të grupuara sipas fjalive. Procesi më kritik është *align_labels_with_tokens()* që trajton problemin e tokenizimit të nënfjalëve. Kur tokenizer-at ndajnë një fjalë në nënfjalë, vetëm nënfjala e parë merr etiketën origjinale, ndërsa të gjithë të tjerët etiketohen me -100 që injorohen gjatë llogaritjes së humbjes.

5.3.3. Arkitekturat

Modeli BERT implementon një arkitekturë të avancuar për detektimin e entiteteve të emëruara në tekst. Ky implementim përdor modelin bazë "*bert-base-multilingual-cased*", i cili është një variant shumëgjuhësor që mbështet përpunimin e teksteve në gjuhë të ndryshme.

Në thelb të arkitekturës qëndron *AutoModelForTokenClassification*, një komponent i specializuar që përdor strukturën e BERT-it për klasifikimin në nivel tokeni. Modeli është i ndërtuar mbi arkitekturën transformer me encoder dykahësh, që do të thotë se ai lexon tekstin në të dyja drejtimet - nga e majta në të djathtë dhe nga e djathta në të majtë - për të kuptuar kontekstin e plotë të çdo fjale.

Arkitektura përmban disa komponentë kryesorë. Tokenizer-i (*AutoTokenizer*) është përgjegjës për ndarjen e tekstit në njësi më të vogla të quajtura token, duke përfshirë edhe nën-fjalët. Kjo është e rëndësishme sepse BERT-i përdor një qasje të quajtur *WordPiece* tokenization, e cila mund të ndajë fjalët e panjohura në pjesë më të vogla që modeli i njeh. Figura 5.7. paraqet arkitekturën e BERT:

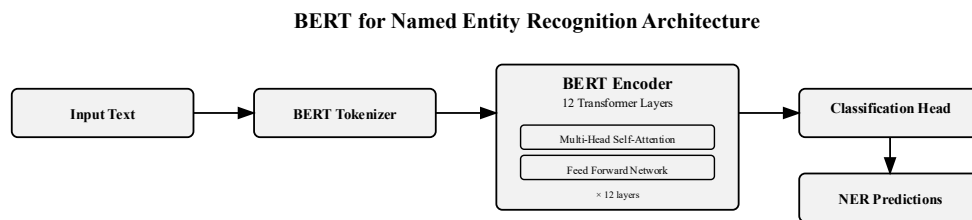


Figura 5.7. Arkitektura e BERT

Një nga sfidat kryesore të arkitekturës është përputhja e etiketave me token-et. Meqenëse BERT-i mund të ndajë një fjalë në disa token, është e nevojshme të përputhen etiketat me token-et e duhur. Kodi implementon një mekanizëm sofistikuar përmes metodës "*align_labels_with_tokens*", ku etiketat speciale (-100) përdoren për token-et që nuk duhet të kontribuojnë në humbjen gjatë trajnimit.

Modeli është i konfiguruar për të punuar me sekuenca deri në 128 token (*max_length=128*), një zgjedhje që balanceon mes performancës dhe efikasitetit. Përdorimi i *DataCollatorForTokenClassification* siguron që të gjitha sekuenat të kenë të njëjtën gjatësi përmes padding-ut, ndërsa attention mask-at sigurojnë që modeli të mos i kushtojë vëmendje token-eve të padding-ut.

Ky dizajn arkitekturor kombinon fuqinë e reprezentimeve kontekstuale të BERT-it me teknika të specializuara për klasifikimin e token-eve, duke krijuar një sistem efikas për njohjen e entiteteve të emëruara në tekste shumëgjuhësore.

Modeli RoBERTa përfaqëson një evolucion të rëndësishëm të arkitekturës së BERT-it. Implementimi përdor "*xlm-roberta-base*", një variant shumëgjuhësor që është trajnuar në një korpus masiv tekstesh nga 100 gjuhë të ndryshme, duke e bërë atë veçanërisht të përshtatshëm për aplikime ndërgjuhësore.

Në thelb të arkitekturës qëndron *XLmRobertaForTokenClassification*, një komponent i specializuar që ndërton mbi përmirësimet arkitekturore të RoBERTa-s. Ndryshe nga BERT-i, RoBERTa eliminon objektivin e NSP dhe përdor një qasje më të sofistikuar për maskimin e token-eve gjatë pre-trajnitimit. Kjo rezulton në një model më efikas që arrin performancë më të mirë me resurse të njëjta kompjuterike.

Arkitektura përdor të njëjtin mekanizëm dykahësh transformer si BERT-i, por me disa përmirësime kyçe. RoBERTa trajnohet duke përdorur një qasje dinamike për maskimin e token-eve në vend të maskimit statik që përdor BERT-i. Kjo qasje dinamike do të thotë se secili token mund të maskohet në mënyra të ndryshme në epoka të ndryshme trajnimi. Figura 5.8. paraqet arkitekturën e përdorur:

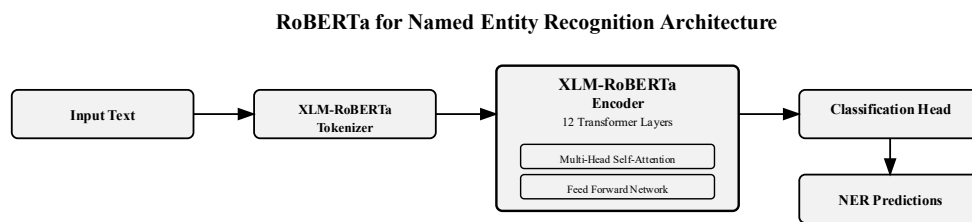


Figura 5.8. Arkitektura e RoBERTa

Tokenizimi në RoBERTa përdor algormin *SentencePiece* në vend të *WordPiece* që përdor BERT-i. Kjo qasje është më efikase për gjuhë të ndryshme dhe krijon një fjalor më të balancuar që funksionon mirë për tekste shumëgjuhësore.

Një ndryshim i rëndësishëm arkitekturor është se RoBERTa nuk përdor token-e segmentimi (segment embeddings) që përdor BERT-i. Në vend të kësaj, RoBERTa mbështetet vetëm në pozicion embeddings dhe token embeddings, duke thjeshtuar arkitekturën dhe duke e bërë atë më fleksibël për sekuenca të gjata.

Modeli ruan të njëjtën strukturë për përputhjen e etiketave me token-et si BERT-i, duke përdorur të njëjtin mekanizëm me etiketat speciale (-100) për token-et që nuk kontribuojnë në humbjen e trajnimit. Megjithatë, mekanizmi i attention-it në RoBERTa është i optimizuar më mirë, duke përdorur një implementim më efikas që redukton kompleksitetin kompjuterik.

Një përparësi e rëndësishme arkitekturore e RoBERTa-s është stabiliteti i saj gjatë trajnimit. Modeli është më pak i ndjeshëm ndaj hiperparametrave dhe shfaq konvergencë më të qëndrueshme, veçanërisht për detyra me të dhëna të kufizuara. Kjo e bën RoBERTa-n një zgjedhje të preferuar për aplikime praktike ku stabiliteti i trajnimit është kritik.

Kjo arkitekturë e RoBERTa-s përfaqëson një balancë të shkëlqyer mes performancës së lartë dhe efikasitetit kompjuterik, duke e pozicionuar atë si një zgjedhje ideale për detyra të ndryshme të përpunimit të gjuhës natyrore, veçanërisht në kontekste shumëgjuhësore.

5.3.4. Struktura e Trajnimit dhe Optimizimi

TrainingArguments konfigurohet me parametra të optimizuar për NER: *learning rate* $2e-5$ [9], *warmup steps* 500, *weight decay* 0.01. BERT dhe RoBERTa trajnohen për 5 epoka *eval_strategy="steps"* dhe *eval_steps=200* sigurojnë vlerësim të rregullt gjatë trajnimit. *load_best_model_at_end=True* dhe *metric_for_best_model="entity_f1"* garantojnë që modeli më i mirë ruhet bazuar në performancën e Entity F1.

5.3.5. Ndryshimet Arkitekturore midis BERT dhe RoBERTa

BERT përdor *AutoModelForTokenClassification* me model-in **bert-base-multilingual-cased** që është i trajnuar në 104 gjuhë duke përfshirë shqipen. RoBERTa implementon *XLNetRobertaForTokenClassification* me **xlm-roberta-base** që përdor arkitekturën RoBERTa të optimizuar dhe është i trajnuar në një korpus më të madh multigjuhësor.

5.3.6. Procesi i Trajnimit dhe Krahimit

Trajnuesi *Trainer* koordinon të gjithë procesin për të dyja modelet duke përdorur *DataCollatorForTokenClassification* për batch-imin e saktë. Pas përfundimit të trajnimit, të dyja modelet vlerësohen në test set dhe ruhen në direktori të ndara. Rezultatet kthehen si fjalorë që përmbajnë emrin e modelit, Token F1, Entity F1 dhe kohën e trajnimit. Funkcionet e analizës krijojnë tabela krahasuese që tregojnë se cili model performon më mirë në kontekstin specifik të NER-it për gjuhën shqipe.

5.4. Rezultatet

Rezultatet e përfutuara prej eksperimenteve janë paraqitur në tabelat e mëposhtme, të ndara për secilin lloj eksperimenti.

5.4.1. Rezultatet e Modeleve Machine Learning

Tabela 5.2. Rezultatet e Modeleve Machine Learning

Modeli	Token F1	Entity F1	Koha e Trajnimit (s)
Naive Bayes	0.909041	0.000000	0.35
Naive Bayes (POS+LEMMA)	0.669424	0.086729	0.88
Extra Trees	0.961089	0.800389	39.04
Extra Trees (POS+LEMMA)	0.961612	0.803670	34.57
Random Forest	0.961157	0.799843	118.72
Random Forest (POS+LEMMA)	0.961514	0.801727	84.1
XGBoost	0.916408	0.261521	141.89
XGBoost (POS+LEMMA)	0.925177	0.429927	149.73
Gradient Boosting	0.759894	0.177133	1380.91
Gradient Boosting (POS+LEMMA)	0.664216	0.220943	1508.86
MLPS	0.907590	0.000000	219.05
MLPS (POS+LEMMA)	0.908704	0.000334	108.81
MLPM	0.908748	0.000000	434.67
MLPM (POS+LEMMA)	0.908631	0.000670	256.55
MLPL	0.908992	0.000000	1054.19
MLPL (POS+LEMMA)	0.909241	0.000000	473.02

Rezultatet e Tabelës 5.2 tregojnë një ndarje të qartë midis algoritmeve në bazë të performancës së tyre. **ET** dhe **RF** dalin si performuesit më të mirë me Entity F1 që arrin 80%, duke treguar aftësi të fortë në identifikimin e entiteteve. Këto algoritme përfitojnë nga kombinimi i shumë pemëve të vendimit, ku ET përdor një mënyrë më të rastësishme në zgjedhjen e ndarjeve, duke rezultuar në trajnim më të shpejtë (34-39 sekonda) krahasuar me Random Forest (84-118 sekonda).

NB tregon një sjellje interesante ku arrin Token F1 të lartë (0.909) por Entity F1 praktikisht zero. Kjo ndodh sepse algoritmi trajton çdo token në mënyrë të pavarur dhe nuk kap varësitë sekuenciale që janë kritike për identifikimin e entiteteve të kompletuar në skemën BIO. Shtesa e veçorive POS+LEMMA dëmton performancën e tij (Token F1 bie në 0.669), duke sugjeruar që kompleksiteti shtesë nuk përshtatet me supozimin e pavarësisë së Naive Bayes.

XGBoost dhe **GB** tregojnë performancë të mesme, ku XGBoost arrin Entity F1 0.261-0.429 dhe GB 0.177-0.220. GB kërkon kohë shumë më të gjatë trajnimi (1380-1508 sekonda) por nuk arrin performancë më të mirë, duke treguar efikasitet të dobët për këtë detyrë.

Shtimi i veçorive POS dhe LEMMA tregon efekte të përziera. Për **ET** dhe **RF**, veçoritë shtesë japin përmirësime të vogla (2-4 pikë në Entity F1) dhe trajnim më të shpejtë, duke sugjeruar që informacioni gramatikor dhe leksikor ndihmon. **XGBoost** përfiton nga veçoritë (Entity F1 nga 0.261 në 0.429), ndërsa **GB** tregon përmirësim të lehtë por me kosto shumë më të lartë kohore. Përmirësimet në GB të Entity F1 janë të papërfillshme pasi modeli dështon në më shumë se 80% të rasteve, dhe Token F1 është ulur.

Të gjitha variantet e **MLP** tregojnë një dështim sistematik në identifikimin e entiteteve, me Entity F1 praktikisht zero pavarësisht nga arkitektura. Kjo tregon se rrjetet feed-forward nuk janë të përshtatshme për detyra sekuenciale si NER, pasi nuk kanë mekanizma për të kapur varësitë midis token-ave fqinje. Token F1 mbetet i lartë (rreth 0.908-0.909) sepse shumica e token-ave në dataset janë O, dhe modelet mësojnë të parashikojnë këtë klasë dominuese.

Rezultatet konfirmojnë se algoritmet tree-based janë më të përshtatshëm për NER kur përdoren veçori të kodifikuara manualisht. ET dhe RF arrijnë balancën më të mirë midis saktësisë dhe efikasitetit, duke treguar se kompleksiteti i lartë nuk është gjithmonë i nevojshëm për rezultate të mira.

Rezultatet e eksperimentit vetëm për NER, përkrijnë afërsisht me rezultatet e [1] për të njëjtat modele, sepse eksperimenti i tyre është përsëritur.

5.4.2. Rezultatet e BiLSTM-ve

Tabela 5.3. Rezultatet e Modeleve BiLSTM

Modeli	Token F1	Entity F1
BiLSTM	0.9778	0.8268
BiLSTM (POS)	0.9814	0.8590
BiLSTM (POS + LEMMA)	0.9818	0.8642
BiLSTM-CRF	0.9786	0.8534
BiLSTM-CRF (POS)	0.9815	0.8712
BiLSTM-CRF (POS + LEMMA)	0.9818	0.8724

Rezultatet tregojnë një përmirësim dramatik në performancë krahasuar me algoritmet klasike. **BiLSTM** bazë arrin Entity F1 0.8268, që është afërsisht 3% më lart se performuesi më i mirë nga algoritmet e thjeshtë. Ky ndryshim thelbësor reflekton aftësinë e arkitekturave të rrjeteve neutrale për të kapur varësitë sekuenciale që janë kritike në NER.

BiLSTM-CRF tregon përparësi të qartë mbi BiLSTM-in e thjeshtë, ku versioni bazë arrin Entity F1 0.8534 krahasuar me 0.8268.

Shtimi i veçorive tregon një progresion të qartë në performancë. Për **BiLSTM-CRF**, Entity F1 rritet nga 0.8534 (bazë) në 0.8712 (me POS) dhe më tej në 0.8724 (me POS+LEMMA). Ky përmirësim gradual tregon se informacioni gramatikor dhe morfologjik kontribuon në identifikimin më të saktë të entiteteve, megjithëse përfitimet janë më të vogla se sa tek algoritmet klasike.

BiLSTM i thjeshtë tregon të njëjtin model: Entity F1 rritet nga 0.8268 (bazë) në 0.8590 (me POS) dhe 0.8642 (me POS+LEMMA). Këto përmirësime janë më të dukshme se tek BiLSTM-CRF, duke sugjeruar se kur nuk ka shtresën CRF, veçoritë shtesë bëhen më kritike për performancën e modelit.

Token F1 mbetet i lartë për të gjitha modelet (0.9778-0.9818), me ndryshime minimale. Kjo konsistencë tregon se të gjitha arkitekturat janë të afta në klasifikimin e saktë të tokenave individuale, por ndryshimi real qëndron në aftësinë për të identifikuar entitetet e tjera.

5.5.3. Rezultatet e Transformerave

Tabela 5.4. Rezultatet e Modeleve Transformer

Modeli	Token F1	Entity F1	Koha e Trajnimit (s)
BERT	0.9861	0.90742	305.34
RoBERTa	0.98596	0.90601	336.85

Rezultatet tregojnë një përparim të konsiderueshëm në performancën e NER-it krahasuar me të gjitha modelet e mëparshme. **BERT** arrin Entity F1 0.9074, që është rreth 4% më lart se performuesi më i mirë nga modelet BiLSTM-CRF me veçori (0.8724). **RoBERTa** arrin Entity F1 0.9060, praktikisht në të njëjtin nivel performance. Ky përmirësim dramatik reflekton fuqinë e modeleve transformer të paratrajnuara që kanë mësuar nga korpuse të mëdha shumëgjuhëshe.

Një aspekt i rëndësishëm është se këto rezultate arrihen pa përdorimin e veçorive shtesë POS dhe LEMMA, ndryshe nga modelet BiLSTM që kishin nevojë për këto veçori për të arritur performancën maksimale. BERT dhe RoBERTa kanë mësuar të ekstrahojnë automatikisht informacion gramatikor dhe morfologjik nga vetë konteksti, duke eliminuar

nevojën për anotim manual të veçorive. Kjo është një avantazh i madh praktik sepse redukton kompleksitetin e preprocessing-it dhe bën modelet më të përgjithshëm.

Token F1 për të dyja modelet është praktikisht identik (0.9861 për BERT, 0.98596 për RoBERTa), duke arritur nivelin më të lartë të parë në të gjithë eksperimentet. Kjo konsistencë e lartë tregon se të dyja modelet janë të afta plotësisht për klasifikimin e token-ave individualë.

Në fund u krye një provë për parashikimin e tag-eve në një fjali dhe rezultatet e modelit, Figura 5.9.

“Shqipëria do të marrë pjesë në Samitin e BE-së në Bruksel të hënën në 20 Maj 2020.”

Word	NER

Shqipëria	B-VEND_1
do	O
të	O
marrë	O
pjesë	O
në	O
Samitin	B-EVENT
e	O
BE-së	B-ORG
në	O
Bruksel	B-VEND_0
të	B-DATE_1
hënën	I-DATE_1
në	O
20	B-DATE_0
Maj	I-DATE_0
2020.	I-DATE_0

Figura 5.9. Rezultati i Parashikimeve

KREU VI

PËRFUNDIME

Ky studim përmirëson burimet për NER në gjuhën shqipe, duke kontribuar me një dataset të ri të kombinuar dhe një vlerësim gjithëpërfshirës të modeleve të ndryshme. Rezultatet mbështesin tezën se arkitekturat *Transformer* të paratrajnuara tejkalojnë dukshëm metodat klasike për NER në shqip, duke arritur një Entity F1 prej 90.7%, krahasuar me 80.4% të algoritmeve tradicionale më të mira apo 87.2% të RNN.

Studimi rivlerëson gjithashtu rolin e arkitekturave RNN, duke treguar se BiLSTM-CRF me veçori të pasura arrin 87.2% Entity F1, duke ofruar një kompromis të vlefshëm midis performancës dhe efikasitetit. Kjo është veçanërisht e rëndësishme për aplikime praktike ku Transformer mund të jenë shumë të kushtueshëm në terma të burimeve.

Një rezultat i rëndësishëm që del nga studimi është se hierarkia e performancës në NER ndjek një model të parashikueshëm: *Pre-trained Transformers* > *BiLSTM-CRF* > *BiLSTM* > *ET dhe RF* > *algoritmet e tjera*. Kjo hierarki reflekton aftësinë e modeleve për të kapur varësitë sekuenciale dhe informacionin kontekstual, duke ofruar një kornizë teorike për zgjedhjen e duhur në detyra të ngjashme.

Studimi gjithashtu hap horizonte të reja kërkimi për NLP në gjuhën shqipe, duke treguar se edhe modelet më të mira aktuale lënë ende hapësirë për përmirësime në performancën e entiteteve. Kjo ngre pyetjen nëse optimizimi i ardhshëm duhet të fokusohet në zgjerimin e të dhënave apo në inovacione arkitekturore.

Nga pikëpamja praktike, kontributi kryesor i këtij studimi është krijimi i një vlerësimi për NER në gjuhën shqipe duke shtuar veçori shtesë, duke vendosur metrika të qarta të performancës që do të mundësojnë krahasime të ardhshme.

Në të ardhmen, një drejtim i rëndësishëm mbetet trajnimi i modeleve Transformer duke shtuar veçori shtesë që deri më tani nuk janë përfshirë në trajnim. Po ashtu, pasurimi i dataset-it me më shumë larmishmëri dhe me një numër më të madh fjalësh do të kontribuonte në përmirësimin e performancës së përgjithshme.

Një rast i mundshëm do të ishte përdorimi i veçorive të tjera gjuhësore të cilat nuk janë integruar në këtë studim por janë pjesë e dataset-it të krijuar. Gjithashtu, mund të eksplorohet përdorimi i modeleve më të mëdha me qëllim që të krahasohet ndikimi i tyre në performancën e detyrës së NER-it në gjuhën shqipe.

Bibliografia

- [1] Kote, N., Kalliri, K., Kalliri, K., Haveriku, A., Muraku, B., & Kajo Meçe, E. (2025). *NER for Albanian Language: A Manually Annotated Corpus and Machine Learning Models*.
- [2] Kote, N., Biba, M., Kanerva, J., Rönqvist, S., and Ginter, F.: Morphological Tagging and Lemmatization of Albanian: A Manually Annotated Corpus and Neural Models. ArXiv, abs/1912.00991 (2019)
- [3] Aroca-Ouellette, S., & Rudzicz, F. (2020). *On Losses for Modern Language Models*. arXiv preprint arXiv:2010.01694.
- [4] Kovaleva, O., Romanov, A., Rogers, A., & Rumshisky, A. (2019). *Revealing the Dark Secrets of BERT*. arXiv preprint arXiv:1908.08593.
- [5] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv preprint arXiv:1810.04805.
- [6] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692*.
- [7] Clark, K., Luong, M.-T., Le, Q. V., & Manning, C. D. (2020). *Electra: Pre-training Text Encoders as Discriminators Rather Than Generators*. arXiv preprint arXiv:2003.105
- [8] Wikipedia contributors. (2025, July 12). *Needleman–Wunsch algorithm*. In *Wikipedia, The Free Encyclopedia*. Retrieved September 02, 2025, from https://en.wikipedia.org/wiki/Needleman–Wunsch_algorithm
- [9] H. Amit, "A gentle introduction to RoBERTa," *Medium*, Nov. 13, 2024. [Online]. Available: <https://medium.com/@heyamit10/a-gentle-introduction-to-roberta-1f1f9babb62d>.
- [10] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997. doi: 10.1109/78.650093
- [11] Huang, Z., Xu, W., & Yu, K. (2015). Bidirectional LSTM-CRF models for sequence tagging. *arXiv preprint arXiv:1508.01991*. <https://arxiv.org/abs/1508.01991>